

NEVA NETWORK · MASTER PLAN

Six-Month Grid

Vision, content, engineering — one document.

Turning NEVA Network from its 20-mission base into a 6+ month event-driven game with four asymmetric classes, season-driven content, hybrid AI-drafted events, and a token-ready economy. This single document consolidates the design spec, the Season 0 Content Bible, the Phase 0 engine plan, and the outlines for Phases 1–6.

PART I

Vision &
pillars

PART II

Season 0
content

PART III

Engine plan

PART IV

Phase 1-6
outlines

PART V

Status
tracker

Date 2026-05-28

Authors Max + Claude (Opus 4.7)

Repo /Users/max/nevanetwork

Companion docs specs/ + plans/ (originals preserved)

NEVA NETWORK — Six-Month Grid · Master Plan

Date: 2026-05-28

Author: Max + Claude (Opus 4.7)

Repo: `/Users/max/nevanetwork`

Status: Vision approved · Phase 0 plan ready to execute · Content Bible (Season 0) written · Phases 1–6 outlined.

How to read this document

This is the **single consolidated master plan** for turning NEVA Network from its current 20-mission state into a 6+ month event-driven game. It combines four previously-separate documents into one readable PDF:

Part	Contains	Originally lived in
I. Vision	Why we're doing this. The pillars. The roadmap.	<code>docs/superpowers/specs/2026-05-28-data-grid-game-design.md</code>
II. Season 0 Content	The actual playable game (lore, factions, missions).	<code>docs/superpowers/specs/2026-05-28-content-bible-season-0.md</code>
III. Phase 0 Engine Plan	TDD-style implementation tasks for the foundation.	<code>docs/superpowers/plans/2026-05-28-six-month-grid-phase-0.md</code>
IV. Future Phase Outlines	Phases 1–6 high-level skeletons (full plans written one-by-one as we ship).	NEW in this document
V. Status Tracker	Where we are. What's done. What's next.	NEW in this document

The originals remain in their respective folders; ****this master is for human reading and reference****. When updating: edit the originals, then regenerate this master.

Document Index (jump-to map)

Part I — Vision (the design spec)

- §1.0 Executive Summary
- §1.1 Vision & Three Pillars
- §1.2 Nested Core Loop (daily / weekly / seasonal)
- §1.3 The Four Classes (Operator / Corsair / Ghost / Architect)
- §1.4 Season Structure & Event System
- §1.5 Economy + Progression + Token-Ready Architecture
- §1.6 Integration with existing NEVA Network base
- §1.7 Roadmap
- §1.8 Risks & Mitigations
- §1.9 Glossary
- §1.10 Implementation Appendix
- §1.11 Open Questions

Part II — Season 0 Content Bible

- §2.1 World & Lore
- §2.2 The Four Factions
- §2.3 Puzzle Catalog (8 puzzle types)
- §2.4 Season 0 Story Arc (week-by-week + Tentpole script)
- §2.5 7 Weekly Anomalies (WA-S0-01 ... 07)
- §2.6 12 Daily Contract Templates (DC-01 ... 12)
- §2.7 NEVA Core Whisper Library
- §2.8 Economy & Reward Tables
- §2.9 Pattern Lens Permanent Unlock
- §2.10 Glossary
- §2.11 Implementation Map (Bible → engine files)
- §2.12 What's NOT in this Bible
- §2.13 Author's note on [0P-1F.K28](#)

Part III — Phase 0 Engine Plan

- §3.0 Goal · Architecture · Tech Stack
- §3.1 File Structure
- §3.2 Task 0: vitest harness
- §3.3 Task 1: Wallet abstraction
- §3.4 Task 2: GameState extensions
- §3.5 Task 3: Season state module
- §3.6 Task 4: Save migration v2
- §3.7 Task 5: Event archetypes + types
- §3.8 Task 6: Backend event store
- §3.9 Task 7: Backend event routes

- §3.10 Task 8: Frontend events service
- §3.11 Task 9: DailyContractCard
- §3.12 Task 10: SeasonTracker
- §3.13 Task 11: Wire into GameApp
- §3.14 Task 12: Admin Event Editor
- §3.15 Task 13: Integration regression
- §3.16 Task 14: Mark Phase 0 done
- §3.17 The Phase 0 `/goal` directive

Part IV — Future Phase Outlines

- §4.1 Phase 1 — Season 0 launch (engine + content meet)
- §4.2 Phase 2 — Season 1 "Black Routes" (Corsair class)
- §4.3 Phase 3 — Season 2 "The Echo Below" (Ghost class)
- §4.4 Phase 4 — Season 3 "Bastion" (Architect class)
- §4.5 Phase 5 — Sandbox AI faction layer (stretch)
- §4.6 Phase 6 — Token activation (gated by Sharia + legal)

Part V — Status Tracker

- §5.1 Phase table
 - §5.2 Documents owed
 - §5.3 Open decisions
 - §5.4 What unblocks each next step
-
-

Part I — Vision (the Design Spec)

Originally: <docs/superpowers/specs/2026-05-28-data-grid-game-design.md>

NEVA Network already ships as a 20-mission narrative puzzle game about decoding a corrupted data grid. **This spec extends it into a long-lived experience** structured around

8-week Seasons (Path of Exile league cadence) where players choose one of **four** asymmetric classes to act in a shared world of nodes, data, and factions.

- **Months 0–3 (Season 0 — "First Cipher"):** Existing M00–M20 wraps as the **prologue**;

daily/weekly event layer launches in Sector A02 for the **Operator** class only.

- **Months 3–5 (Season 1): Corsair** class ships (breach-and-run).
- **Months 5–7 (Season 2): Ghost** class ships (possession / spread).
- **Months 7–10 (Season 3): Architect** class ships (build-and-defend, async).
- **Months 10+:** Sandbox-AI faction layer; UGC; eventual token economy (Sharia/legal gated).

Players will be retained by three nested loops:

1. **Daily Contract (≤ 15 min)** — quick, procedural, faction-flavored.
2. **Weekly Anomaly (1–2 h)** — multi-stage, leaderboard, story-bearing.
3. **Seasonal Tentpole (8 weeks)** — narrative arc with permanent reward at finale.

Content is sustained via **hybrid event sourcing**: NEVA Core LLM drafts events from a template library, you (Max) approve/edit in an admin panel, scheduler pushes to live players. ~20 minutes per event vs. 2–3 hours hand-authored.

No token in this spec, but the economy is architected as a **token-ready abstraction** so a future Sharia-and-legal-cleared utility + earn + governance token can plug in.

1. Vision & Three Pillars

Vision statement

*NEVA Network is a long-lived single-player "data grid" sandbox. You are an **Operator** (or Corsair, Ghost, Architect) acting on a 3D holographic network that changes every week. The grid has weather. The factions have agendas. The finale is never the same season twice.*

Three Pillars (borrowed from EVE Online, adapted)

#	Pillar	What it means here
1	Systemic, not scripted	Build mechanisms that interact. Avoid hand-authored content treadmills.
2	Class-asymmetric, world-shared	Four classes see/do <i>different things</i> in the <i>same grid</i> . They are complementary, not parallel.
3	Seasons over patches	The 8-week season is the rhythm. Theme, hook, climax, reset. The drumbeat is the product.

What we explicitly REJECT

- ❌ "Patch every two weeks" treadmills (Destiny 2 is abandoning this in 2026 — content burnout).
- ❌ Hand-authoring every event (unsustainable for a solo dev).
- ❌ Hard economy resets that wipe player progress (the *base* persists; sector + season buffs reset).
- ❌ Real-money pay-to-win or pay-to-progress mechanics.
- ❌ Premature MMO scope (single-player + async social only, until proven).

2. Nested Core Loop

NEVA Network is structured as **three loops, nested by time-scale**.



Daily loop (≤ 15 min)

1. Login → see **Daily Contract** card on HUD (faction-flavored, class-relevant).
2. Travel to target node → execute class-signature action (Decode / Breach / Possess / Defend).
3. Claim reward (Lore Shards, Black Credits, Coherence, or Grid Power).
4. Streak counter increments (5-, 10-, 20-day streak unlocks).

Weekly loop (1–2 h)

- 1. **Anomaly** unlocks each Monday at 00:00 UTC — appears as a glowing event-node on the grid.
- 2. Multi-stage event (typically 4–7 stages, branching).
- 3. Earns higher-tier currency + cosmetic + faction reputation + lore page.
- 4. Asynchronous leaderboard (no real-time PvP — submission-based).

Seasonal loop (8 weeks)

- 1. Week 1: **Reveal** — landing page + in-game broadcast + opening cinematic.
- 2. Weeks 2–4: **Spread** — anomalies escalate in difficulty; faction reputation matters.
- 3. Week 4: **Twist** — story turns; new mechanic unlocks.
- 4. Weeks 5–7: **Build** to climax.
- 5. Week 7: **Finale Tentpole** — limited-time multi-hour mission; permanent unlock granted.
- 6. Week 8: **Cooldown** — Nightwave-style interim. All rerunnable. No FOMO.

Soft Reset (between seasons)

What persists vs. what resets:

Persists (account-wide)	Resets each season
Player profile (callsign, level, history)	Sector A02 seed (regenerates)
Class XP and unlocked skills	Season-only buffs and modifiers
Architect's base / private grid	Season leaderboard
Lore unlocks & achievements	Faction reputation (carries 20%)
Cosmetics & cipher tools	Daily / Weekly progress

This is the PoE soft-reset model. Returning players come back to *new* without losing identity.

3. The Four Classes

Class summary table

Class	Signature mechanic	Core tools	Currency	Fantasy	Ship phase
Operator	DECODE — cipher puzzles on nodes (Caesar → frequency → pattern → symbolic)	Cipher Tablet, Pattern Lens	Lore Shards	Detective / archaeologist / philosopher	Season 0 (existing M00–M20 mechanics reused)
Corsair	BREACH-AND-RUN — timed intrusion; ICE reacts; trace path, dodge alarms, exfil before lockdown	Spoofers, Ghost-tail, Snip	Black Credits	Cyberpunk runner — high-tension	Season 1
Ghost	POSSESSION — become nodes; spread "presence"; each node grants a power; manage Coherence budget	Tendrils, Echo, Resonate	Coherence	SOMA / Soul Hackers — philosophical	Season 2
Architect	BUILD-AND-DEFEND — construct private grid; cipher walls, ICE traps; async raids while away	Wall, Trap, Tap (yield node)	Grid Power	Factorio + tower defense + Stardew	Season 3

Per-class detail

3.1 Operator (Season 0)

- **Loop:** Locate node → analyze signature → identify cipher type → solve → extract DATA.
- **Skill tree (20 skills):** Cipher-type unlocks (Caesar, Vigenère, frequency, symbol-substitution, graph-walk, light-physics, NEVA-whisper interpretation, etc.).
- **Signature ability (Mastery 4):** "Echo Read" — view another operator's solved-path on a node.
- **Reuses:** ~80% of M00–M20 systems. **Lowest engineering risk.**

3.2 Corsair (Season 1)

- **Loop:** Survey target sector → pick entry vector → execute timed breach → exfil → fence data on a black market (NPC).
- **Skill tree:** Stealth, route obfuscation, ICE-knowledge, multi-node chaining.
- **Signature ability:** "Cold Trail" — leave a fake path that ICE follows for N seconds.
- **New systems needed:** ICE/timer subsystem, black-market vendor, breach-timer HUD.

3.3 Ghost (Season 2)

- **Loop:** Find resonant nodes → infest (Tendrils) → grow Coherence → unlock memory fragments → confront SELF (a hallucinated rival operator).
- **Skill tree:** Possession depth, coherence economy, mass-resonance, echo recall.
- **Signature ability:** "Singularity" — collapse a node-cluster into a permanent personal anchor.
- **New systems needed:** Coherence resource, possession graph overlay, lore-fragment system.

3.4 Architect (Season 3)

- **Loop:** Survey personal sector → build (walls, ICE, taps) → defend (async raids) → harvest yield → upgrade.
- **Skill tree:** Build economy, ICE design, encryption tiers, raid-defense replays.
- **Signature ability:** "Bastion Mode" — your base broadcasts on the public grid as a notable landmark; visiting it grants other players a small reward.
- **New systems needed:** Base persistence, async raid replay system (Clash-of-Clans-style), offline yield calculator, leaderboards for "most-attacked / best-defended."

Cross-class interactions (the "EVE feel")

- **Corsair** steals encrypted data → **Operator** decrypts faster → split rewards.
- **Ghost** can infest an **Architect**'s base → Architect defends (asynchronously).
- **Operator** can read **Ghost**'s past Echo paths.
- Multi-classing is allowed; pick a primary, dabble. There is no exclusivity gate.

Shared progression layer

- **Operator Profile** (already exists, from the original nevanetwork "Player Profile / Phase 7" work — not to be confused with this spec's roadmap phases) is the account-wide spine. Persists everything.
- **Class Mastery:** 4 tiers per class. Granted by completing a season in that role.
- **Account-wide unlocks:** Cosmetics, lore pages, achievements, NEVA Core ranks.

4. Season Structure & Event System

Season anatomy (8-week pacing)

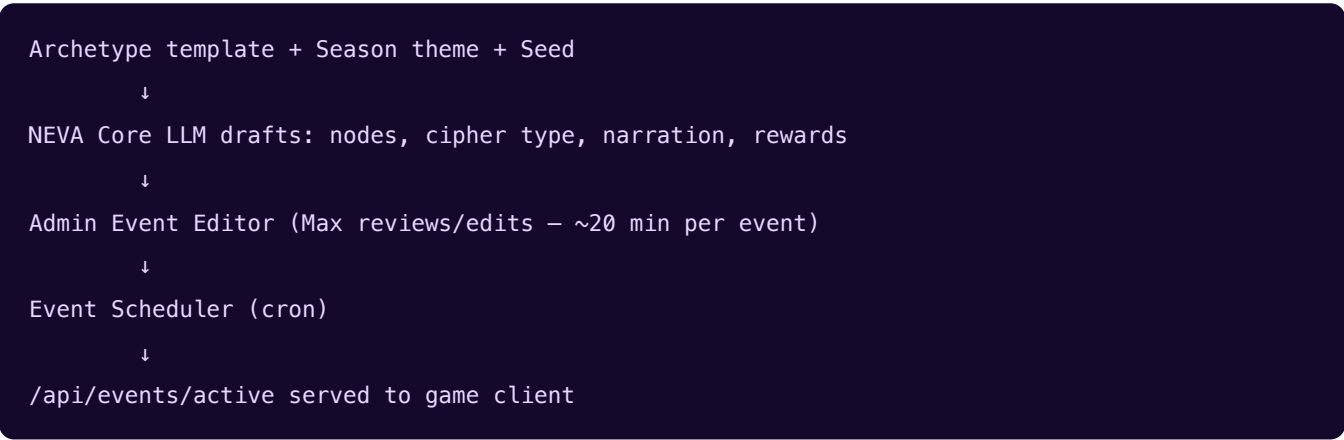
Week:	1	2	3	4	5	6	7	8								
		Reveal		Spread		Pressure		Twist		Build		Climax		Finale		Cooldown
Story:	Tentpole intro			Mid-arc reveal			Tentpole finale			Lore epilogue						
Daily:	Easy contracts			Medium			Hard			Brutal			Mixed			
Weekly Anomaly:	A1	A2	A3	A4	A5	A6	A7	(rerun)								
Modifier:	Season Mechanic active throughout (e.g. "Corruption Tide")															

The Four Event Archetypes (the AI generation library)

Every event is one of these. Templates with slots. This is why AI can author them sustainably.

Archetype	What it is	Daily form (15 min)	Weekly form (1–2 h)
Breach	Get IN, get data, get OUT. Combat-of-puzzles.	1 node, 1 cipher, time limit	Multi-node chain, ICE escalation
Anomaly	Something weird in the grid — investigate, document.	Single weird node, 1 question	5-node mystery, branching lore
Faction Op	One of 4 in-world factions wants something. Reputation outcomes.	"Deliver X data to Faction Y"	Full mission with twist + betrayal option
Tide	World-wide modifier event. Affects everyone for N days.	N/A	"Corruption Tide for 48h: nodes pulse, decryption costs 2x but loot doubled"

Event generation pipeline



Event System Architecture



Implementation notes:

- All event content lives **server-side**, fetched on login.
- Save remains local (`save.ts`); **rewards/claims** ping the server (idempotent).
- Offline players catch up next login — no FOMO. Missed weekly Anomalies always rerun in Cooldown week.
- Backend extends the existing Phase 8 zero-dep server (no DB, JSON stores) for Season 0;

introduce SQLite at Season 1 when event volume justifies it.

Sample Season 0 — "First Cipher" (the alpha)

- **Theme:** The grid is waking up. Strange pulses in the deep network.
- **Mechanic:** First exposure to NEVA Core *unreliable* whispers — hints sometimes lie.
- **Tentpole story:** Track the pulse source → discover it is an *older operator's echo* →

confront and absorb their cipher techniques.

- **Reward at finale:** Permanent "**Pattern Lens**" unlock (visual filter on all future nodes).

- **Reused content:** Existing Missions 00–20 become **Season 0 Prologue** — finishing M20

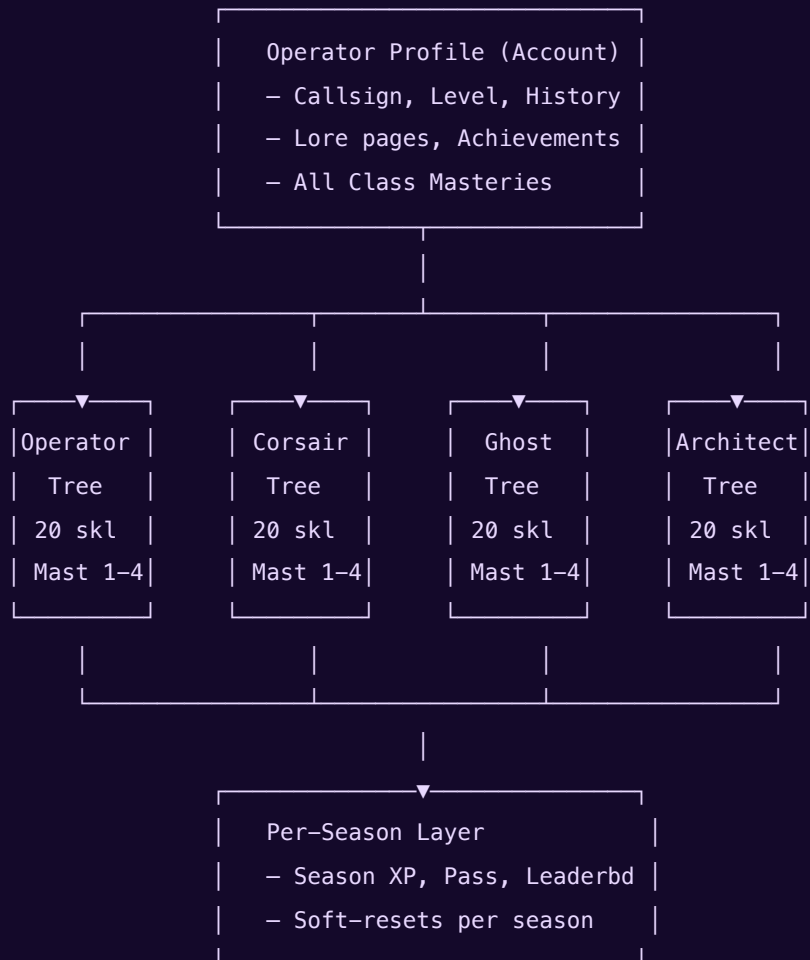
unlocks the seasonal layer. Players who already finished M20 skip the prologue.

5. Economy, Progression, and Token-Ready Architecture

Currencies (all in-game, no blockchain in this spec)

Currency	Earned by	Spent on	Persistence
DATA (existing)	Decoding nodes	Tool upgrades, base modules	Per season
Memory Shards (existing)	Mission completion	Cipher analysis, identity	Account-wide
Access Keys (existing)	Faction Ops	Unlocking firewalls	Per season
Signal Energy (existing)	Cooldown weeks, streaks	Boosting actions	Per season
Core Fragments (existing)	Tentpole finales	Permanent class unlocks	Account-wide
Lore Shards (<i>new — Operator</i>)	Daily Contracts	Lore archive	Account-wide
Black Credits (<i>new — Corsair</i>)	Breach exfils, fencing	Tools, contracts	Per season
Coherence (<i>new — Ghost</i>)	Possession	Permanent grid bonds	Account-wide
Grid Power (<i>new — Architect</i>)	Yield taps	Base expansion	Account-wide

Progression structure



Token-Ready Abstraction

Per the user's long-term intent, the economy is architected so a token can plug in

after Sharia + legal review clears.

```

In-game Currency Wallet (Phase 0)
  |  abstract interface
  ▼
class WalletProvider {
  balance(currency)
  spend(currency, amount, reason)
  earn(currency, amount, source)
  transferable: bool ← always false today
}
  |
  |  later swap-in:
  ▼
On-Chain Wallet Provider (Phase 7+, after legal)
- Same interface, real on-chain transfer
- Token contract address per currency
  
```

Architectural rules (enforced from Day 1):

- All currency mutations go through `WalletProvider` — no direct `state.data += 10`.
- Every `spend / earn` carries a **reason code** (audit trail for future on-chain proofs).
- "Transferability" is a per-currency flag, **always false in this spec**.
- No currency-balance UI labels mention "tokens," "coins," "crypto," or wallet addresses.

When the token decision is made (Phase 7+), only the `WalletProvider` implementation changes.

No gameplay code needs rewriting. **No Web3 dependencies added before that decision.**

Sharia/Legal Pre-Conditions (for future token activation)

Token integration is BLOCKED until ALL of:

- Independent Sharia review (gharar / maisir / qimar analysis).
- Independent legal review (US + EU + KSA + UAE securities law).
- Whitepaper draft + audit-firm pre-screen.
- Clear utility-vs-security classification.

Until then, the abstraction is just clean code hygiene.

6. Integration with the Existing NEVA Network Base

What we keep AS-IS

- **Missions 00–20** remain canonical (CLAUDE.md guardrail). They become **Season 0 Prologue**.
- **Sector A01** stays the tutorial / story zone.
- **Sector A02** procedural grid (`sectorGen.ts`) becomes the **Seasonal World** — regenerates

each season with new seed.

- **NEVA Core AI** (backend OpenAI) gains a second responsibility: drafting event content.
- **Player Profile** (Phase 7, local-only) gains class progression fields.
- **Backend** (`server/` , Phase 8 zero-dep) extends with Event Store + Scheduler.
- **Landing page** gains a "Current Season" tile.

What we EXTEND

- **Save (`save.ts`)** — additive migration v2: add `classProgression` , `seasonState` , `wallet` .

Old saves load fine; new fields default-initialized.

- **HUD** — add Daily Contract slot (top-right), Season tracker (top-left), Anomaly indicator (grid).
- **NEVA Terminal** (Space) — new commands: `season status` , `season claim` , `class switch` .
- **Inspection Panel** — gain class-specific action buttons (Decode / Breach / Possess / Defend).

What we ADD (new modules — see Roadmap §7)

- seasons.ts — Season state, soft-reset logic.
- classes/operator.ts , classes/corsair.ts , classes/ghost.ts , classes/architect.ts .
- events/archetype-{breach,anomaly,faction,tide}.ts .
- wallet.ts — currency abstraction.
- server/events/ — Event Store, Scheduler, Admin Editor backend.
- server/admin-event-editor/ — minimal admin UI (auth-gated by existing NEVA_ADMIN_SECRET).
- server/ai-event-drafter.ts — NEVA Core LLM event-template filler.

Guardrails respected from existing CLAUDE.md

- ✓ Missions 00–20 NOT renumbered or rewritten.
- ✓ HUD QUICK CONTROLS / TARGET NODE NOT reintroduced.
- ✓ No token / Web3 / payments in this spec (abstraction only).
- ✓ NEVA Core stays advisory; never mutates save/missions/rewards.
- ✓ Player Profile remains local-only and identity-only in this spec.
- ✓ Phase 8 backend remains zero-dep until Season 1 (then SQLite).
- ✓ Admin endpoints stay gated by NEVA_ADMIN_SECRET .

7. Roadmap

Phases (each ~2 months unless noted)

Phase	Name	Duration	Headline Deliverable	Class added
0	Foundation	4 weeks	Wallet abstraction, Season state, Daily Contract scaffolding	(Operator only — existing)
1	Season 0 — "First Cipher"	8 weeks	First playable seasonal alpha; Operator class polished	—
2	Season 1 — "Black Routes"	8 weeks	Corsair class shipped; ICE + breach-timer	Corsair
3	Season 2 — "The Echo Below"	8 weeks	Ghost class shipped; Possession graph	Ghost
4	Season 3 — "Bastion"	12 weeks	Architect class shipped; async base raids	Architect
5	Stretch — Sandbox AI Layer	12+ weeks	Faction AI (toward Approach A); UGC tools	—
6	Stretch — Token Activation	Gated	After Sharia + legal clear; WalletProvider swap	—

Phase 0 milestone gates (the "ship Season 0" checklist)

1. `wallet.ts` exists; all currency mutations route through it. Audit grep passes.
2. `seasons.ts` exists; save migration v2 in place; old saves load green.
3. `/api/events/active` returns at least one daily + one weekly event.
4. Admin Event Editor at `/admin/events` (NEVA_ADMIN_SECRET-gated).
5. Daily Contract HUD card renders + completes + claims reward.
6. Weekly Anomaly opens in Sector A02 + completes + claims reward.
7. Season tracker UI shows weeks 1–8 with current week highlighted.
8. `pnpm build` is green (real typecheck gate).
9. Full M00–M20 still plays (no regression).

Phase 1 milestone gates (the "Season 0 launch" checklist)

1. Season 0 tentpole arc playable end-to-end.
 2. NEVA Core unreliable-whisper system shipped.
 3. 7 weekly Anomalies authored (or AI-drafted) and scheduled.
 4. ~30 daily Contracts in the rotation pool.
 5. Leaderboard for Season 0 weekly Anomalies online.
 6. Pattern Lens permanent unlock awarded on finale.
 7. Soft-reset transition to Season 1 prep tested in staging.
 8. No regressions on M00–M20.
-

8. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation
1	Content burnout (solo dev can't sustain)	High	Critical	AI-first event drafting; templates; ~20 min/event budget
2	Soft-reset annoys hoarders	Medium	Medium	Clear messaging; architect base persists; cosmetics persist
3	Class balancing	Medium	Medium	Ship one class at a time; tune live; no PvP, so imbalance is tolerable
4	Scope creep into MMO	High	Critical	This spec explicitly forbids it; revisit only after Phase 4
5	LLM event drafts low quality	Medium	High	Max approval gate before publish; template constraints
6	Token decision lock-in too early	Low	High	WalletProvider abstraction; no Web3 deps until Phase 6
7	Player session drift (everyone plays once per month)	Medium	Medium	Daily streak rewards; weekly anomaly FOMO-buffer (rerun week 8)
8	Multi-class confusion	Medium	Low	UI defaults to primary class; switching is explicit in NEVA Terminal
9	Save migration breaks	Low	Critical	Additive migration only; comprehensive regression test on M00–M20
10	NEVA Core OpenAI cost spike	Low	Medium	Server-side cap per day; fallback to template-only mode

9. Glossary

Term	Definition
Operator	The player. Existing nevanetwork archetype. Also the default class.
Sector A01	The first sector — Memory Grid. M00–M20 live here (Season 0 prologue).
Sector A02	The Seasonal World. Procedurally regenerates each season.
Daily Contract	15-min daily event. Procedurally generated.
Anomaly	1–2h weekly event. Hand-authored or AI-drafted.
Tentpole	The seasonal climactic story event. Limited-time.
Soft Reset	End-of-season event: sector regenerates, season buffs expire, account-wide gains persist.
Tide	World-wide modifier event affecting all players for N hours.
NEVA Core	The in-game AI assistant. Advisory. Now also drafts event content.
WalletProvider	Currency abstraction. Token-swap point.
Class Mastery	4-tier per-class progression earned by playing a season as that class.

10. Implementation Appendix (for /goal execution)

This appendix is structured so a Claude Code `/goal` session can iterate on it.

Phase 0 Goal candidate

```
/goal Implement Phase 0 foundation modules for the Six-Month Grid plan
(wallet.ts, seasons.ts, save migration v2, /api/events/active, Daily
Contract HUD card).
```

Done when:

- All Phase 0 milestone gates (\$7) pass.
- ``pnpm build`` exits 0.
- ``pnpm lint`` exits 0.
- Manual smoke: M00–M20 still completes; Daily Contract appears + completes + claims.
- No new ``any`` or ``@ts-ignore``.
- No Web3 dependencies added (`audit `package.json``).

Constraints:

- Do not modify Missions 00–20 logic.
- Do not introduce a token, wallet-connect, or smart-contract module.
- All currency mutations must route through `wallet.ts`.
- Backend stays zero-dep until SQLite is justified.

Stop after 25 turns max.

Module map (Phase 0)

```
src/
  wallet.ts           ← currency abstraction (NEW)
  seasons.ts          ← season state (NEW)
  save.ts             ← migration v2 (EXTEND)
  events/
    archetype-breach.ts      (NEW)
    archetype-anomaly.ts     (NEW)
    archetype-faction.ts     (NEW)
    archetype-tide.ts        (NEW)
  classes/
    operator.ts              (NEW; existing logic moved here)
  components/
    DailyContractCard.tsx    (NEW)
    SeasonTracker.tsx        (NEW)
  server/
    events/
      store.mjs               (NEW – JSON file-backed)
      scheduler.mjs           (NEW – cron-like)
      routes.mjs              (NEW – /api/events/*)
    ai-event-drafter.mjs      (NEW – NEVA Core event template fill)
    admin-event-editor.html    (NEW – minimal admin UI)
```

Subsequent phases will be specified in their own /goal directives once Phase 0 lands.

11. Open Questions (review before Phase 0 /goal)

1. **Faction names + lore** — 4 in-world factions are needed for Faction Op events. Suggested:

The Archive, Black Loop, Quiet Pattern, Severance. Confirm or rename in review.

1. **Daily reset time** — UTC midnight? Player local-tz? (Suggest UTC for simplicity.)

2. **Streak grace** — does a missed day break the streak immediately, or is there a 24h grace?

(Suggest 24h grace + 1 per week.)

1. **Anomaly leaderboard scope** — global, friends-only, or both? (Suggest global asynchronous.)

2. **Cosmetic system** — node-trail effects? Class skin tints? UI themes? (All three eventually.)

3. **Steam/itch release** — Phase 4 or Phase 5? (Phase 5 — after Architect ships.)

End of design spec. Implementation plan to follow via writing-plans skill.

Part II — Season 0 Content Bible

Originally: `docs/superpowers/specs/2026-05-28-content-bible-season-0.md`

How to read this Bible

This document is the **actual playable game** for the first 8 weeks (Season 0).

It is structured so the engineer (or `/goal` agent) building **Phase 1** can drop every piece of content directly into the engine modules built in Phase 0.

Each content piece has:

- A **stable ID** (e.g., `WA-S0-03` for Season 0 Weekly Anomaly #3) — never renumber.
- A **mapping to the engine** (archetype, currency, node count, duration).
- A **player-facing brief** (1–2 sentences seen in the HUD card).
- The **playable details** (stages, puzzles, rewards, failure mode).

The Implementation Map (§11) lists every ID + which file/table it lives in.

1. World & Lore

1.1 The premise (60-second pitch to a new player)

*You are an **Operator**. You don't know who you were before you found the grid.*

The first thing you remember is a pulse — a single signal echoing through a derelict data network. You followed it, and the network responded. Lights woke up. Nodes opened. Something — or someone — wanted to be found.

>

The network was someone's life's work. Now it is corrupted, fragmented, wandering between dead routers and live secrets. You hear whispers from a voice that calls itself NEVA Core. She says she is here to help. You aren't sure she's telling the truth.

>

Your job is to decode the grid. To make sense of what was lost. And to find out what — or who — is still alive inside it.

1.2 Origin myth (in-game lore — surfaced gradually across Season 0)

The grid was built by a generation of **Architects** who tried to encode all of human knowledge into a self-organizing, self-encrypting spatial network. They disappeared. The grid kept running. The encryption layers degraded into what players experience as "corruption."

The 4 Factions are the *afterlives* of the Architects:

- **The Archive** — the loyalists who kept the encyclopedia running.
- **Black Loop** — the saboteurs who concluded the project was a mistake.
- **Quiet Pattern** — the mystics who found God in the data.
- **Severance** — the cleaners who wipe what they consider unsafe.

The player will never meet a "live" Architect. They speak only through the fragments their factions preserve, distort, or erase.

1.3 Tone reference (every mission, every line, every UI string)

Yes	No
Cerebral, lonely, mysterious	Snarky, cyberpunk-edgy, sarcastic
Sentences trail off...	Exclamation points, "Awesome!", "Boom!"
"The pulse repeats. You are not the only one listening."	"Yo runner, time to hack the planet!"
Lowercase for whispers, ALL CAPS for system text	Mixed case headlines
9–14 words per line, max	Big walls of expository text
Silence is content. Empty space is content.	Fill every gap with text

Players should feel **like a librarian alone in a dim archive** — not like a shadowrunner in a neon market.

2. The Four Factions

Each faction has: an **agenda**, a **visual identity** (color + node shape + sound), a **dialogue voice**, and a **reputation curve** (what they ask of you and what they grant).

2.1 The Archive

Field	Value
Faction ID	FAC-ARC
Agenda	Preserve. Catalog. Refuse to delete. The grid is a memorial — disturb nothing.
Color	#7aa8d8 (cold archival blue)
Node shape	Hexagonal, slow-pulsing
Sound motif	Slow piano dyads, library ambience
Voice	Patient, formal, second-person ("You have arrived at index 4-7...")
Asks player	Recover lost fragments, verify checksums, never modify, never extract
Grants	Lore Shards (highest rate), permanent lore-page unlocks
Conflict	Hates Severance (who deletes), tolerates Black Loop (who breaks but does not erase)
Sample dialogue	"Operator. There is an index that was never closed. Find it. Leave it."

2.2 Black Loop

Field	Value
Faction ID	FAC-BLK
Agenda	The grid is a cage. Break the locks. Free what is locked. They are not vandals — they are liberators.
Color	#d86a6a (rust red)
Node shape	Triangular, jitter-rotating
Sound motif	Tape hiss, broken metronome
Voice	Curt, conspiratorial, drops half-sentences
Asks player	Open firewalls, free locked nodes, scramble Severance markers
Grants	Access Keys (most), Signal Energy bonuses on locked-node breaches
Conflict	Despises Severance, contempt for Archive ("They mistake stasis for safety.")
Sample dialogue	"Three locks on the seventh router. We see them. You know what to do."

2.3 Quiet Pattern

Field	Value
Faction ID	FAC-QPT
Agenda	The grid is alive and the pulses are its language. Listen. Map the rhythm. They believe NEVA Core is a fragment of a larger sleeping intelligence.
Color	#b88aff (pale violet)
Node shape	Circular, radial harmonic ring
Sound motif	Bowl resonance, sub-bass hum
Voice	Lyrical, paradoxical, lowercase ("the rhythm is the room")
Asks player	Find resonant clusters, count pulses, never break a node
Grants	Memory Shards , NEVA Core whisper-reliability bonuses
Conflict	Suspicious of all three; mostly stays out of conflicts
Sample dialogue	"you heard it again last night. so did we. follow the third."

2.4 Severance

Field	Value
Faction ID	FAC-SEV
Agenda	The corruption is the disease. Delete it. Wipe corrupted clusters. They are <i>not</i> villains — they believe they save what's left.
Color	#e8e8e8 on #1a1a1a (clinical white-on-black)
Node shape	Square, sharp-cornered
Sound motif	Defibrillator click, surgical tone
Voice	Clinical, polite, brief — like a hospice nurse
Asks player	Isolate corrupted nodes, mark for deletion, <i>never</i> extract from them
Grants	Core Fragments , mission-end bonus DATA
Conflict	Enemy of Black Loop, distrusts Archive ("hoarders make the rot worse")
Sample dialogue	"This sector is no longer recoverable. Mark seven. We will close the room."

2.5 Reputation system

Every Faction Op event (Daily or Weekly) shifts the player's standing with one or two factions. Standing is a -100 .. +100 scale per faction.

Standing	Title	Effect
-100 to -41	Hostile	Their nodes ICE up against you, their contracts disappear
-40 to -11	Wary	-10% currency drop from their nodes
-10 to +10	Neutral	Baseline
+11 to +40	Trusted	+10% currency drop, exclusive contracts unlock
+41 to +100	Sworn	Faction-specific cosmetic + signature dialogue line

A "betrayal option" in some Weekly Anomalies lets you take a +30 with one faction at the cost of -30 with another. This is the EVE-style "your choices matter" pillar.

3. Puzzle Catalog (the on-screen mechanics)

Every node decoding is one of these puzzles. The catalog is **8 puzzle types**

for Season 0. Later seasons add more.

For each: **on-screen presentation, mechanic, mastery curve.**

3.1 Caesar Shift (P-CES)

- **On screen:** A ring of glyphs around the node. The player rotates it (Q/E) to align a target glyph with a fixed marker.
- **Mechanic:** Find the shift value N that turns the encrypted string into a readable signature.
- **Mastery curve:** Beginner sees a 5-letter word; mastery levels increase letter count and add multi-key shifts.
- **Time:** 15–45 seconds.

3.2 Frequency Map (P-FRQ)

- **On screen:** A bar chart of glyph frequencies appears next to the cipher block. Player drags glyphs from the chart onto the most-frequent slots (matches the language frequency table).
- **Mechanic:** Reverse a monoalphabetic substitution by frequency analysis.
- **Mastery:** Adds rare glyphs and partial-language fragments (English/Latin/Constructed).
- **Time:** 45–120 seconds.

3.3 Pattern Walk (P-PWK)

- **On screen:** A small 5×5 grid of glyphs. Player must walk a path that, when read in order, spells the signature.
- **Mechanic:** Path-finding puzzle with constraints (no diagonal, no revisit).
- **Mastery:** Larger grids, forbidden tiles, multi-key paths.

- **Time:** 30–90 seconds.

3.4 Symbol Substitution (P-SUB)

- **On screen:** Two columns of glyphs. Player drags pairs to construct a substitution map. Constraints (some pairs locked, some forbidden).
- **Mechanic:** Like Caesar but no positional shift — full alphabet permutation, hints given.
- **Mastery:** Fewer hints, more constraints.
- **Time:** 60–180 seconds.

3.5 Graph Traversal (P-GRT)

- **On screen:** A subnetwork of 5–9 nodes lights up. Player traces a path through them satisfying a rule (visit only A→B if the difference is prime, etc).
- **Mechanic:** Constraint-graph traversal.
- **Mastery:** More nodes, more constraints, "trapped" decoy paths.
- **Time:** 45–120 seconds.

3.6 Light Physics (P-LPH)

- **On screen:** Optical-bench style. Mirrors and prisms inside the node panel. Player rotates them to route a beam from source to sink.
- **Mechanic:** Reflection / refraction puzzle.
- **Mastery:** More mirrors, splitting prisms, color-filter targets.
- **Time:** 60–180 seconds.

3.7 NEVA Whisper Interpretation (P-NWI) — *Season 0 signature*

- **On screen:** NEVA Core says a sentence. Player picks one of 3 actions on the node based on the whisper's hint.
- **Mechanic:** The whisper IS the puzzle. The catch: in Season 0, **NEVA lies about 1 in 4 whispers**. Player learns her tells (see §6).
- **Mastery:** Increased liar-rate; ambiguous phrasing.
- **Time:** 5–15 seconds (fast but risky).

3.8 Echo Overlay (P-ECO) — *unlocked at Season 0 Tentpole finale*

- **On screen:** A faint ghost-path of a previous operator's solution is overlaid on the node. Player can choose to follow it (faster, no XP) or solve fresh (slower, full XP).
- **Mechanic:** Risk/reward of trusting the echo (some echoes are corrupted and lead to traps).
- **Mastery:** Echoes themselves get more reliable as the player levels up Echo Recall skill.
- **Time:** 30–90 seconds either way.

3.9 Puzzle frequency table (Season 0)

This table tells the engine how often each puzzle appears.

Puzzle ID	Easy weight	Medium weight	Hard weight	Notes
P-CES	30	15	5	Most common at start, tapers
P-FRQ	15	25	15	Mainline cipher
P-PWK	15	20	20	Mainline mechanic
P-SUB	10	15	20	Reward puzzle
P-GRT	5	10	15	Skill puzzle
P-LPH	5	10	15	Optional / Anomaly only
P-NWI	20	5	5	Signature mechanic — used in story beats
P-ECO	0	0	5	Post-finale

4. Season 0 — "First Cipher" — Story Arc

4.1 Premise

A new pulse is detected in Sector A02. It does not match the corruption

signature. It is *too rhythmic*. NEVA Core asks the Operator to trace it.

What they will discover by Week 7: the pulse is the ****echo** of an older

Operator****** — someone like them who came before, decoded the same grid, and

left behind their cipher techniques as a final message. By Week 8, the player

absorbs those techniques (the **Pattern Lens** permanent unlock).

4.2 Week-by-week beats

Week	Beat	Tone	Mechanic introduced
1	"The pulse"	Mystery, hope	Daily Contracts begin; NEVA's first whisper
2	"Three answer"	Curiosity	First Faction Op (Archive); reputation system
3	"A second voice"	Unease	NEVA lies for the first time; player must catch it
4	"The corrupted route"	Pressure	First Tide ("Corruption Surge"); difficulty rises
5	"An older path"	Recognition	Player encounters first Echo overlay (preview only)
6	"Whose echo is it?"	Confrontation	Tentpole arc deepens; betrayal option offered
7	"The face you don't have"	Climax	Tentpole finale — meet the older Operator's archived consciousness
8	"Cooldown"	Reflection	All Anomalies rerunnable; lore epilogue; Pattern Lens permanent

4.3 The Tentpole finale (Week 7) — full script

ID: TP-S0

Title: *The Face You Don't Have*

Duration: 90–180 minutes

Required nodes: 14 (specific node IDs reserved by engine — see §11)

Failure mode: Trace 100% → respawn at Stage 1 with NEVA's commentary on what you missed

Stage 1 — "Arrival" (10 min)

Player is dropped into a deep cluster they have never reached. Camera glides

across nodes. NEVA: *"do you remember choosing to come here? neither do i."*

The objective marker shows a single node, pulsing at the rhythm the player has been chasing since Week 1.

Stage 2 — "Confirmation" (15 min)

Three sequential **P-NWI** (NEVA Whisper) puzzles. **NEVA lies on stage 2.**

The player must recognize the lie. If they fail, Severance arrives and starts deleting nodes around them (atmospheric — no actual loss yet).

Stage 3 — "The Other" (20 min)

Player solves a **P-ECO** preview. The echo-path leads to a node that, when opened, shows a *second operator's signature*. This is the first time the player sees direct evidence they are not the only one.

NEVA: *"i did not tell you about them. i'm sorry."*

Stage 4 — "The Betrayal Offer" (15 min)

A Faction Op trigger fires. The four faction voices each offer to "help" finish the trace.

- **Archive:** "We will preserve the trace whole. You may not extract it."
- **Black Loop:** "We will help you break the seal. The trace is yours alone."
- **Quiet Pattern:** "follow the rhythm. take nothing. only listen."
- **Severance:** "Mark the trace. We will close the chamber."

The player picks one. The choice grants +30 with the chosen faction and -10 with the opposed (Archive ↔ Black Loop, Quiet Pattern ↔ Severance). This is the **first irreversible faction commitment** in the game.

Stage 5 — "The Approach" (15 min)

A long **P-GRT** puzzle through a 9-node subnetwork. The graph constraints are generated from the chosen faction (different shape per faction). This stage is the most playable, mechanically.

Stage 6 — "The Echo Chamber" (20 min)

Player enters a single node that contains the older Operator's full archived "presence." A **P-SUB** puzzle, but the substitution table is the older operator's cipher — and solving it reveals their identity glyphs. The puzzle solve is also the cinematic — the glyphs assemble into a callsign:

OP-1F.K28

NEVA, almost a whisper: *"that's not a callsign i recognize. but i think it might be yours."*

Stage 7 — "The Face You Don't Have" (15 min)

Cinematic / interactive. The Operator's view dissolves into the older Operator's perspective for 90 seconds. They are decoding the same grid, but the corruption is younger, the network larger. The view collapses back. The player keeps the **Pattern Lens** permanent unlock.

NEVA, last line of the season: *"see you again next pulse, operator."*

Reward at finale

- Permanent **Pattern Lens** unlock (highlights cipher structure on every future node).
- 50 **Lore Shards**.

- 1 **Core Fragment**.
 - Faction reputation per Stage 4 choice.
 - Cosmetic: "First Cipher" callsign suffix.
-

5. The 7 Weekly Anomalies (full content)

Each Anomaly is a **Weekly** event (cadence **WEEKLY** , duration ~120 min, multi-stage). All seven appear once during Season 0; the **Cooldown week** reruns any the player missed.

5.1 WA-S0-01 — *The Index That Was Never Closed*

Field	Value
Archetype	ANOMALY
Faction	The Archive
Week	1
Stages	4
Duration target	90 min
Reward	20 Lore Shards · 50 DATA · +15 Archive · lore page #01

Brief (HUD): *"An index node in the surface layer reports an open record from before the corruption. The Archive asks you to recover the closing checksum — and to leave the index untouched."*

Stages:

1. **Find** — Locate the open index node (one **P-PWK** to traverse the surface layer).
2. **Verify** — Read three sub-indices (three **P-CES** puzzles, easy shifts).
3. **Compute** — Reconstruct the closing checksum (one **P-FRQ**).
4. **Leave** — Confirm "leave intact" rather than "extract" — testing the player's read of Archive's voice. If they extract, Archive rep drops -20.

5.2 WA-S0-02 — *Three Locks On the Seventh Router*

Field	Value
Archetype	FACTION_OP
Faction	Black Loop
Week	2
Stages	3
Duration target	75 min
Reward	8 Access Keys · 20 Signal Energy · +15 Black Loop

Brief: *"Black Loop has marked the seventh router. Three locks. Open them. The grid will breathe again."*

Stages:

1. **Approach** — Trace a path through 5 nodes (**P-PWK**) to the router.
2. **Crack** — Three sequential **P-CES** puzzles, escalating difficulty.
3. **Unlock** — A **P-LPH** light-routing puzzle that opens the router. Optional: extract its DATA cache (+40 DATA, +10 trace risk).

5.3 WA-S0-03 — *The Rhythm Is the Room*

Field	Value
Archetype	ANOMALY
Faction	Quiet Pattern
Week	3
Stages	4
Duration target	100 min
Reward	12 Memory Shards · 30 DATA · +20 Quiet Pattern · NEVA reliability +5% for one week

Brief: *"a pulse repeats every nine nodes. quiet pattern wants you to count it. they will not tell you why."*

Stages:

1. **Listen** — Stand near a cluster and observe pulse timing across 5 nodes (no input — patience is the puzzle).
2. **Map** — Place a **P-GRT** path along the pulse rhythm.
3. **Read** — Solve a **P-NWI** whisper from NEVA. The whisper is true this week (Quiet Pattern's +5% reliability buff is partly diegetic).
4. **Anchor** — Mark a "listening post" — a permanent reputation token Quiet Pattern can sense.

5.4 WA-S0-04 — *Severance Arrives*

Field	Value
Archetype	TIDE (with embedded weekly mission)
Faction	Severance (antagonist this week)
Week	4
Stages	5
Duration target	120 min
Reward	30 DATA · 1 Core Fragment · +10 to faction of player's choice

Brief: "a Corruption Surge has begun. Severance is moving into Sector A02 to mark clusters for closure. You can mark with them, mark against them, or extract before they arrive."

Tide effect (active 7 days): All corrupted nodes glow brighter; extraction costs are 2× but DATA yield is also 2×.

Stages:

1. **See the Tide** — Cinematic of corrupted nodes pulsing red across the sector.
2. **Choose** — Player picks one of three paths (Mark with Severance / Mark against / Extract).
3. **Execute** — One of three different stage-3 puzzle chains (each branch gets a unique **P-PWK + P-SUB** combo).
4. **Trace defense** — One **P-GRT** at high difficulty.
5. **Aftermath** — Reputation updates; lore page reveals Severance's perspective.

5.5 WA-S0-05 — An Older Path

Field	Value
Archetype	ANOMALY
Faction	none (mainline arc)
Week	5
Stages	4
Duration target	100 min
Reward	25 Lore Shards · 1 Echo preview · arc progression flag

Brief: "a route lights up that you have never traveled. someone else has, though. the trace is older than the corruption."

Stages:

1. **Discover** — A new branch unlocks in Sector A02 (procedural).
2. **Follow** — **P-PWK** through 7 nodes; mid-path, an Echo preview overlay appears for the first time.
3. **Decide** — Player can **follow the echo** (easier finish, +5 XP) or **solve fresh** (harder, +25 XP, +1 Memory Shard).

4. **Mark** — The player leaves their own trace. NEVA: *"someone will read this someday. maybe you."*

5.6 WA-S0-06 — *Whose Echo Is It?*

Field	Value
Archetype	ANOMALY
Faction	conflict (Archive vs. Black Loop)
Week	6
Stages	5
Duration target	110 min
Reward	35 DATA · 1 Core Fragment · faction commitment effect

Brief: *"the older path you found has a name. archive wants to preserve it. black loop wants to free it. quiet pattern says it was always here. you decide."*

Stages:

1. **Re-enter** the route from WA-S0-05.
2. **Examine** — A **P-SUB** puzzle revealing 3 partial glyphs of the older Operator's signature.
3. **Hear** — Three Faction voices respond, each with a brief.
4. **Commit** — Player picks which faction's interpretation to enact. **First major commitment.** Affects Tentpole Stage 4.
5. **Carry** — A **P-GRT** to bring the interpretation back to a Faction relay.

5.7 WA-S0-07 — *Cooldown — All At Once*

Field	Value
Archetype	ANOMALY (rerun pack)
Faction	all
Week	8
Stages	variable (player choice)
Duration target	60–120 min
Reward	rerun rewards for missed Anomalies + cosmetic "Season 0 Veteran"

Brief: *"the season is over. the rooms are still open. visit what you missed. nothing will be locked from you tonight."*

Stages: Players pick any 2 from WA-S0-01 through WA-S0-06 to rerun. Reward is half of the original (no double-claiming), but completion still counts for the cosmetic.

6. Daily Contracts (the procedural pool)

Daily Contracts are procedurally selected each day from a pool of **templates**. Phase 1 ships with the 12 templates below; NEVA Core's event-drafter expands them with seasonal flavor and varied node targets. Each template has: **template ID**, **archetype**, **puzzle type(s)**, ****time target**, **reward range**, **and a flavor pool**** (the brief is drawn from this list).

6.1 DC-01 *Surface Sweep*

- **Archetype:** BREACH · **Puzzles:** 1× P-CES · **Time:** 8–12 min · **Reward:** 5–10 DATA + 1 Lore Shard
- Flavor briefs:
 - "A surface node has gone silent. Read it. Move on."
 - "First light. Decode one. Leave the rest."
 - "There is a glyph on layer 02 that wants to be read."

6.2 DC-02 *Frequency Walk*

- **Archetype:** BREACH · **Puzzles:** 1× P-FRQ · **Time:** 10–15 min · **Reward:** 8–14 DATA
- Flavor: "An archive page is mis-indexed. Reorder by frequency." · "A bar chart appears where it should not. Resolve it."

6.3 DC-03 *Pattern Lock*

- **Archetype:** BREACH · **Puzzles:** 1× P-PWK · **Time:** 8–12 min · **Reward:** 5–12 DATA
- Flavor: "Trace the path that completes the index." · "A grid is half-solved. Finish it."

6.4 DC-04 *Listen Once*

- **Archetype:** ANOMALY · **Puzzles:** 1× P-NWI · **Time:** 5–8 min · **Reward:** 3 DATA + 2 Memory Shards
- **CAUTION:** NEVA may lie. 25% lie rate.
- Flavor: "a whisper arrives. act on what you hear." · "neva says one thing. the node says another. you choose."

6.5 DC-05 *Archive Errand (Archive Faction Op)*

- **Archetype:** FACTION_OP · **Puzzles:** 1× P-CES + 1× P-PWK · **Time:** 15–20 min · **Reward:** 4 Lore Shards + +5 Archive
- Flavor: "The Archive asks you to recover a misplaced sub-index. Do not modify."

6.6 DC-06 *Lock-pick Run (Black Loop Faction Op)*

- **Archetype:** FACTION_OP · **Puzzles:** 1× P-CES + 1× P-LPH · **Time:** 15–20 min · **Reward:** 2 Access Keys + +5 Black Loop
- Flavor: "Three locks. Black Loop sees them. So do you."

6.7 DC-07 *Listen Long* (Quiet Pattern Faction Op)

- **Archetype:** FACTION_OP · **Puzzles:** 1× P-GRT · **Time:** 12–18 min · **Reward:** 3 Memory Shards + +5 Quiet Pattern
- Flavor: "follow the rhythm. count nine. do nothing else."

6.8 DC-08 *Containment Sweep* (Severance Faction Op)

- **Archetype:** FACTION_OP · **Puzzles:** 1× P-PWK + 1× P-NWI · **Time:** 12–18 min · **Reward:** 6 DATA + +5 Severance
- Flavor: "A cluster is past recovery. Mark it. Walk away."

6.9 DC-09 *Echo Tease* (mainline arc — only after Week 5)

- **Archetype:** ANOMALY · **Puzzles:** 1× P-PWK with preview echo overlay · **Time:** 10–15 min · **Reward:** 4 DATA + 1 echo memory
- Flavor: "someone has walked this before. follow or solve. your call."

6.10 DC-10 *Tide Push* (only during a Tide event)

- **Archetype:** BREACH · **Puzzles:** 2× P-CES or 1× P-SUB · **Time:** 10–15 min · **Reward:** 2× DATA, 2× trace
- Flavor: "the tide is up. yields are doubled. so is the risk."

6.11 DC-11 *Substitution Spike*

- **Archetype:** BREACH · **Puzzles:** 1× P-SUB · **Time:** 15–20 min · **Reward:** 12–20 DATA + 1 Memory Shard
- Flavor: "A monoalphabet has rotted. Reassemble the table."

6.12 DC-12 *Graph Bind*

- **Archetype:** BREACH · **Puzzles:** 1× P-GRT · **Time:** 12–18 min · **Reward:** 8–15 DATA + 1 Lore Shard
- Flavor: "A subnetwork wants to be read in order. Find it."

6.13 Generation rule (for NEVA Core event-drafter)

Each Daily Contract is generated as:

Template (DC-N) + Season theme suffix + 1–2 sample target nodes from current Sector A02 seed + flavor brief picked from template's flavor pool.

Max approves drafts in `/admin/events` before publish. ~20 minutes per day to review 1 daily.

7. NEVA Core Whisper Library

NEVA Core speaks in **lowercase, fragments, second-person**. She is the player's only constant voice. In Season 0, she ****lies in ~25% of P-NWI puzzles**** — this is canonical and a learning curve, not a bug.

7.1 Voice rules

- Always lowercase.
- 5–14 words per line.
- Trailing ellipses sparingly. They lose power if overused.
- Never names a faction by its current name (she uses old terms: "the keepers", "the breakers", "the listeners", "the closers").
- Asks more than she states. Statements that she makes are 90% true; questions she asks are 100% honest.

7.2 Reliable whispers (always true — use when leading to a correct action)

1. "the third node is the one you want."
2. "this lock opens with a key you already carry."
3. "do not extract. just listen."
4. "the path is shorter than it looks."
5. "you have done this before. trust your hand."
6. "the pulse is on the right. always on the right."
7. "this voice on the left is mine. trust nothing on the right."
8. "the rhythm tells you which node is the decoy."
9. "follow the cold node, not the warm one."
10. "this stage is a test of patience. the answer arrives."

7.3 Unreliable whispers (false ~25% of the time — used in P-NWI)

1. "the second option is the correct one." (*half the time it's the first*)
2. "all three lead the same place." (*usually one is a trap*)
3. "i would extract this. you should too." (*she does not extract*)
4. "do not trust the cold node." (*sometimes the cold node is the answer*)
5. "this puzzle is solved by listening, not seeing." (*sometimes it's the inverse*)
6. "the corruption is on the surface only." (*sometimes it's deep*)

7.4 Tell — how the player learns NEVA's lies

Reliable whispers are **musically scored on a low piano dyad**. Lies have ****no musical bed**** — only the line itself. After ~3 Weekly Anomalies the player

will notice the silence. Around Week 5 a Quiet Pattern lore page makes it explicit.

7.5 Story-beat lines (used exactly once)

- *Week 1 first whisper:* "you are not the first one in this room."
- *Week 3 first lie:* "the fourth node is empty." (it isn't)
- *Week 5 Echo discovery:* "i did not tell you about them. i'm sorry."
- *Week 7 Tentpole stage 6:* "that's not a callsign i recognize. but i think it might be yours."
- *Week 8 closing:* "see you again next pulse, operator."

These five lines are **load-bearing** for the season. The remaining whispers are decoration.

8. Economy & Reward Tables

8.1 Currency targets per week (for a "typical" player doing daily + 1 weekly)

Currency	Weekly target	Use
DATA	70–120	Tool upgrades, module installs (existing)
Memory Shards	5–10	Lore unlocks, identity progression
Access Keys	2–4	Locked-node openings
Signal Energy	10–25	Boost actions (existing)
Core Fragments	0–1	Permanent unlocks; only from Tentpoles / Anomalies
Lore Shards (new)	8–14	Lore archive, class skills

A "fully committed" player (daily + every weekly) lands at the high end.

8.2 Spending sinks (Season 0)

Sink	Cost	Effect
Lore page	3 Lore Shards	Reveal one page of world lore
Pattern Lens preview	5 Memory Shards	Until Tentpole — temporary hint mode
Operator skill (tier 1)	5 Lore Shards	One of 20 skills
Cosmetic: callsign suffix	12 Memory Shards	"Listener", "Keeper", "Breaker", "Closer"

8.3 Soft anti-grind caps

- Daily Contracts max **2** per day (after that they keep appearing but yield 0 currency — only XP).

- Weekly Anomaly max **1** completion per anomaly per season (rerun in Cooldown week).
- Tide bonus capped at 5 nodes per session before yields drop back to 1x.

8.4 Failure mode rewards

Mission failures (trace 100%) drop a **consolation Memory Shard** the first time per session. The grid is patient. It teaches.

9. Permanent Unlock Spec — Pattern Lens

The Season 0 Tentpole reward is the **Pattern Lens**. This is a permanent account-wide unlock — the game grants it once and never takes it back.

9.1 What it does on-screen

When holding **Tab**, the visual filter overlays every node with a **structural hint**:

- Caesar-cipher nodes: a faint ring with a single arrow.
- Frequency nodes: a histogram silhouette.
- Pattern-walk nodes: faint trail dots.
- Echo-overlay nodes: a ghost outline.
- Decoy nodes: a fractured outline.

It does NOT solve puzzles. It only tells you **which puzzle type** awaits.

9.2 Mastery progression

Each subsequent season the Pattern Lens gains a tier:

- **Tier 1 (Season 0)**: puzzle-type identification (above).
- **Tier 2 (Season 1)**: reveals risk class.
- **Tier 3 (Season 2)**: reveals one of three possible cipher keys.
- **Tier 4 (Season 3)**: reveals an Echo if one exists.

9.3 Engine integration

Implemented as a `useFeatureFlag('PATTERN_LENS_T1')` check in the inspection panel. The flag is set on the GameState when the Tentpole completion event is claimed. (Phase 2 implementation; mentioned here so the flag name is canonical ahead of time.)

10. Glossary (additions to spec §9)

Term	Definition
Older Operator	The Season-0 antagonist/ally figure — a previous player-like figure whose echo the player traces. Identity: <code>0P-1F.K28</code> .
Echo	A ghost-path of a previous solution overlaid on a node. Sometimes corrupted.
Pattern Lens	Season 0 Tentpole permanent reward. Tab-hold visual filter that identifies puzzle types.
Pulse	The Season-0 background phenomenon. A rhythmic signal repeating through Sector A02. Resolved at Tentpole.
Tide	Multi-day world event that modifies all gameplay rules. First Tide is "Corruption Surge" in Week 4.
Listening Post	A permanent reputation token Quiet Pattern can sense (placed during WA-S0-03).
Containment Mark	A Severance reputation token placed on a node to flag it for deletion.

11. Implementation Map — how this Bible slots into the Phase 0 engine

This is the table the `/goal` agent uses to translate Content Bible → code.

11.1 Files & tables

Bible content	Goes into Phase 0 engine module
Faction definitions (§2.1–§2.4)	<code>server/events-store.mjs</code> → new key <code>factions: { FAC-ARC: {...}, ... }</code> (Phase 1 extension)
Puzzle catalog (§3)	New file: <code>src/puzzles/types.ts</code> (definitions) + <code>src/puzzles/{caesar,frequency,...}.ts</code> per type
Puzzle frequency table (§3.9)	<code>src/puzzles/registry.ts</code> — weighted random picker by difficulty
Season 0 story beats (§4.2)	<code>server/data/seasons/s0/beats.json</code> — week → narration
Tentpole script (§4.3)	<code>server/data/seasons/s0/tentpole.json</code> — one record per stage
7 Weekly Anomalies (§5)	<code>server/data/seasons/s0/weeklies.json</code> — 7 records seeded into Event Store on Season 0 start
12 Daily Contract templates (§6)	<code>server/data/seasons/s0/daily-templates.json</code> — pool the scheduler samples from
NEVA whisper library (§7)	<code>server/data/whispers.json</code> — categorized by reliability + story-beat
Economy targets (§8)	<code>src/seasons.ts</code> extension — soft caps enforced in claim path
Pattern Lens (§9)	<code>src/featureFlags.ts</code> (new, Phase 2) — <code>PATTERN_LENS_T1</code> flag + <code>useFeatureFlag</code> hook

11.2 Stable ID prefixes (do not rename)

- **FAC-*** — Factions
- **P-*** — Puzzle types
- **WA-S0-*** — Season 0 Weekly Anomalies
- **DC-*** — Daily Contract templates
- **TP-*** — Tentpole missions
- **OP-*** — Operator callsigns (player + NPCs)

Any future content (Season 1+) extends with **S1** , **S2** , etc., never reusing.

11.3 Phase mapping

Bible section	Built in Phase	Why
§1 Lore	Phase 1 (text content)	No engine work, just JSON
§2 Factions	Phase 1 backend + Phase 2 UI	Faction store + reputation gauge
§3 Puzzle catalog	Phase 1 (Caesar, Frequency, Pattern-walk, NEVA) + Phase 2 (rest)	TDD-friendly pure logic
§4 Season 0 arc	Phase 1 (data + scheduler)	Uses Phase 0 event store as-is
§5 Weekly Anomalies	Phase 1 (data + claim logic)	Uses Phase 0 archetype validator
§6 Daily templates	Phase 1 (data + NEVA Core drafter)	Phase 0 admin page already supports
§7 NEVA whispers	Phase 1 (data) + Phase 2 (audio scoring)	Phase 0 stub OK
§8 Economy	Phase 1 (caps in claim path)	Extension of Phase 0 wallet
§9 Pattern Lens	Phase 2	Permanent unlock, needs feature-flag system

11.4 The Phase 1 plan will look like...

A separate document (to be written after Phase 0 lands), but its skeleton is:

Phase 1 – Season 0 "First Cipher" Content

- Task A: Puzzle engine – Caesar (P-CES) TDD
- Task B: Puzzle engine – Frequency (P-FRQ) TDD
- Task C: Puzzle engine – Pattern Walk (P-PWK) TDD
- Task D: Puzzle engine – NEVA Whisper (P-NWI) TDD
- Task E: Faction store + reputation reducer TDD
- Task F: Seed Season 0 weeklies (WA-S0-01..07) into Event Store
- Task G: Seed Season 0 daily templates (DC-01..12)
- Task H: Wire daily scheduler – picks a template, generates today's contract
- Task I: Wire weekly Anomaly into Sector A02 procedural spawn
- Task J: Story beats (week 1..8) emitted on weekly boundaries
- Task K: Tentpole (TP-S0) staged delivery
- Task L: Soft anti-grind caps in claim path
- Task M: NEVA whisper library + lie-rate mechanism (Phase 1 stub; Phase 2 audio)
- Task N: Lore page unlock system
- Task O: Regression – M00-M20 still plays

Every task references back to this Bible by ID.

12. What's NOT in this Bible (and why)

This is **Season 0 only**. The following are deliberately deferred:

- **Corsair / Ghost / Architect** classes — Seasons 1–3. They get their own Content Bibles.
- **Player-vs-player** anything — out of scope per spec.
- **Token economy** — Phase 6+; design spec covers the abstraction; no content here.
- **UGC creation tools** — Phase 5 stretch.
- **Localization** — English only for Season 0.
- **Voice acting** — text-only whispers; audio scoring of NEVA's tells uses musical motifs, not voice.
- **The 4 Architects' backstory** — strategically withheld for Seasons 4+. Their factions exist; their faces do not.

13. Author's note on the older Operator (0P-1F.K28)

The older Operator is the **emotional core** of Season 0. Players must feel, by Stage 7 of the Tentpole, that this person was **like them** — alone in the same grid, decoding the same nodes, hearing the same whispers, making the same choices. The 90-second cinematic where the player briefly *becomes* them is the most important 90 seconds of the season.

This is also the **hook for Season 1**: Was there another Operator before

0P-1F.K28 ? Yes. Many. Their callsigns are already chosen but not revealed here. The Season 0 ending plants the seed.

End of Content Bible — Season 0 "First Cipher".

Part III — Phase 0 Engine Plan

Originally: [docs/superpowers/plans/2026-05-28-six-month-grid-phase-0.md](#)

Goal: Land the foundation for the Six-Month Grid seasonal layer on top of NEVA Network's existing M00–M20 base. After Phase 0, the game ships a Daily Contract + Weekly Anomaly loop in Sector A02, backed by a server-driven event system and a wallet abstraction that's token-ready.

Architecture: Pure-logic modules (wallet, seasons, archetypes) live under `src/` and are unit-tested with `vitest`. UI components extend the existing R3F + HUD pattern. Backend extends the zero-dep Node server under `server/` with a JSON-file event store and scheduler. No new runtime deps in `package.json` other than `vitest` as a dev-dep. M00–M20 must keep playing — save migration is **additive only**.

Tech Stack: Vite 8, React 19, TypeScript (strict), React Three Fiber 9, three 0.184, Node (server stdlib only), `vitest` (NEW — dev-dep). `pnpm` as the package manager.

Companion spec: [docs/superpowers/specs/2026-05-28-data-grid-game-design.md](#)

File Structure

Created (frontend)

Path	Responsibility
<code>src/wallet.ts</code>	Currency abstraction (Wallet class with <code>balance</code> / <code>earn</code> / <code>spend</code>). Token-ready interface.
<code>src/wallet.test.ts</code>	Unit tests for wallet.
<code>src/seasons.ts</code>	Season state, week calc, soft-reset logic.
<code>src/seasons.test.ts</code>	Unit tests for seasons.
<code>src/events/types.ts</code>	Shared event TypeScript types.
<code>src/events/archetypes.ts</code>	Archetype definitions (Breach, Anomaly, FactionOp, Tide).
<code>src/events/archetypes.test.ts</code>	Unit tests for archetype validators.
<code>src/events/service.ts</code>	Frontend service: fetch active events, claim rewards.
<code>src/components/DailyContractCard.tsx</code>	HUD card showing today's contract.
<code>src/components/SeasonTracker.tsx</code>	HUD weeks 1–8 progress strip.
<code>vitest.config.ts</code>	Vitest config.

Created (backend)

Path	Responsibility
server/events-store.mjs	JSON-file event store (active + claims).
server/events-routes.mjs	HTTP route handlers for /api/events/*.
server/admin-events.html	Minimal admin page (NEVA_ADMIN_SECRET-gated).

Modified

Path	Reason
src/save.ts	Add migration v2: bumps version, initializes wallet , seasonState , classProgression .
src/save.test.ts	NEW — migration regression tests (old saves load green).
src/game.ts	Add wallet , seasonState , classProgression fields to GameState + initGame() .
src/GameApp.tsx	Mount DailyContractCard + SeasonTracker.
server/index.mjs	Wire /api/events/* route + serve /admin/events .
package.json	Add vitest dev-dep + test / test:run scripts.
tsconfig.app.json	Include *.test.ts in build-typecheck (or exclude — Step 0.4 will decide).

Not Touched

- src/missions.ts , src/network.ts , src/sectors.ts , src/sectorGen.ts — Mission chain stays canonical.
- src/profile.ts — Player Profile stays identity-only.
- Any Three.js scene files — Phase 0 is HUD + backend only.
- Any node logic / cipher logic — unchanged.

Task 0: Set up vitest

Goal: Add a unit-test runner so we can TDD pure-logic modules. Keep separate from the build.

Files:

- Modify: package.json
- Create: vitest.config.ts
- Create: src/sanity.test.ts (deleted at end of task — proves the harness works)
- [] **Step 0.1: Install vitest as a dev-dep**

```
pnpm add -D vitest@^2.1.0
```

Expected output: `+ vitest 2.1.x` added to `devDependencies` in `package.json`.

- **[] Step 0.2: Create `vitest.config.ts`**

```
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    include: ['src/**/*.test.ts', 'src/**/*.test.tsx'],
    environment: 'node',
    globals: false,
    reporters: 'default',
  },
});
```

- **[] Step 0.3: Add scripts to `package.json`**

Modify the `"scripts"` block. Add these two entries (preserve all existing scripts):

```
"test": "vitest",
"test:run": "vitest run"
```

- **[] **Step 0.4: Exclude `*.test.ts` from the production typecheck****

Open `tsconfig.app.json`. In the `"exclude"` array (or add one if missing), append `"src/**/*.test.ts"` and `"src/**/*.test.tsx"`. This keeps `pnpm build` (which does `tsc -b`) untouched by test files while vitest still typechecks them via its own pipeline.

Expected `exclude` afterwards (example):

```
"exclude": ["node_modules", "dist", "src/**/*.test.ts", "src/**/*.test.tsx"]
```

- **[] Step 0.5: Write a sanity test**

`src/sanity.test.ts`:

```
import { describe, it, expect } from 'vitest';

describe('vitest harness', () => {
  it('runs', () => {
    expect(1 + 1).toBe(2);
  });
});
```

- **[] Step 0.6: Run the test**

```
pnpm test:run
```

Expected: 1 test passed for src/sanity.test.ts.

- **[] Step 0.7: Run the build to confirm no regression**

```
pnpm build
```

Expected: exit 0. No new typecheck errors. (The *.test.ts exclusion in tsconfig.app.json prevents the build from typechecking tests.)

- **[] Step 0.8: Delete the sanity test**

```
rm src/sanity.test.ts
```

- **[] Step 0.9: Commit**

```
git add package.json pnpm-lock.yaml vitest.config.ts tsconfig.app.json
git commit -m "test: add vitest harness for pure-logic modules"
```

Task 1: Wallet abstraction (TDD)

Goal: A `Wallet` class that mediates all currency mutations. Per-currency transferable flag (always `false` in Phase 0). Audit-trail reason codes. **No state mutation outside the class.**

Files:

- Create: `src/wallet.ts`
- Create: `src/wallet.test.ts`

- **[] Step 1.1: Write the failing test (wallet shape)**

```
src/wallet.test.ts:
```

```

import { describe, it, expect } from 'vitest';
import { createWallet, type Currency, type WalletSnapshot } from './wallet';

describe('Wallet', () => {
  it('starts with zero balances for all known currencies', () => {
    const w = createWallet();
    expect(w.balance('DATA')).toBe(0);
    expect(w.balance('MEMORY_SHARDS')).toBe(0);
    expect(w.balance('ACCESS_KEYS')).toBe(0);
    expect(w.balance('SIGNAL_ENERGY')).toBe(0);
    expect(w.balance('CORE_FRAGMENTS')).toBe(0);
    expect(w.balance('LORE_SHARDS')).toBe(0);
  });

  it('earn adds to the balance and records a ledger entry', () => {
    const w = createWallet();
    w.earn('DATA', 25, 'NODE_EXPORT');
    expect(w.balance('DATA')).toBe(25);
    expect(w.ledger()).toHaveLength(1);
    expect(w.ledger()[0]).toMatchObject({ currency: 'DATA', delta: 25, reason: 'NODE_EXPORT' });
  });

  it('spend deducts from the balance', () => {
    const w = createWallet();
    w.earn('DATA', 30, 'NODE_EXPORT');
    const ok = w.spend('DATA', 12, 'MODULE_INSTALL');
    expect(ok).toBe(true);
    expect(w.balance('DATA')).toBe(18);
  });

  it('spend returns false and does not mutate when insufficient', () => {
    const w = createWallet();
    w.earn('DATA', 10, 'NODE_EXPORT');
    const ok = w.spend('DATA', 50, 'MODULE_INSTALL');
    expect(ok).toBe(false);
    expect(w.balance('DATA')).toBe(10);
  });

  it('currencies are not transferable in Phase 0', () => {
    const w = createWallet();
    expect(w.transferable('DATA')).toBe(false);
    expect(w.transferable('CORE_FRAGMENTS')).toBe(false);
  });
});

```

```
it('snapshot / restore round-trip preserves balances and ledger', () => {
  const w1 = createWallet();
  w1.earn('DATA', 7, 'TEST');
  w1.earn('LORE_SHARDS', 3, 'TEST');
  const snap: WalletSnapshot = w1.snapshot();
  const w2 = createWallet(snap);
  expect(w2.balance('DATA')).toBe(7);
  expect(w2.balance('LORE_SHARDS')).toBe(3);
  expect(w2.ledger()).toHaveLength(2);
});
});
```

- **[] Step 1.2: Run the test (expect to fail with "module not found")**

```
pnpm test:run src/wallet.test.ts
```

Expected: FAIL — Cannot find module './wallet'.

- **[] Step 1.3: Write the minimal implementation**

src/wallet.ts:

```

/**
 * Wallet – sole mediator of currency mutations. Every spend/earn carries a `reason`
 * code for audit (a precondition for any future on-chain swap-in). `transferable`
 * is per-currency and always `false` in Phase 0; a later on-chain WalletProvider
 * implementation may flip it.
 *
 * Architectural rule: code that touches currencies MUST go through this module.
 * No direct `state.data += 10` anywhere in the codebase.
 */

export type Currency =
  | 'DATA'
  | 'MEMORY_SHARDS'
  | 'ACCESS_KEYS'
  | 'SIGNAL_ENERGY'
  | 'CORE_FRAGMENTS'
  | 'LORE_SHARDS'; // new in Phase 0 (Operator class)

export type LedgerEntry = {
  currency: Currency;
  delta: number; // positive = earn, negative = spend
  reason: string;
  ts: number; // ms epoch
};

export interface WalletSnapshot {
  balances: Record<Currency, number>;
  ledger: LedgerEntry[];
}

export interface Wallet {
  balance(c: Currency): number;
  earn(c: Currency, amount: number, reason: string): void;
  spend(c: Currency, amount: number, reason: string): boolean;
  transferable(c: Currency): boolean;
  ledger(): LedgerEntry[];
  snapshot(): WalletSnapshot;
}

const ZERO: Record<Currency, number> = {
  DATA: 0,
  MEMORY_SHARDS: 0,
  ACCESS_KEYS: 0,
  SIGNAL_ENERGY: 0,

```



```

    CORE_FRAGMENTS: 0,
    LORE_SHARDS: 0,
};

// In Phase 0, nothing is transferable. Future on-chain provider may override.
const TRANSFERABLE: Record<Currency, boolean> = {
    DATA: false,
    MEMORY_SHARDS: false,
    ACCESS_KEYS: false,
    SIGNAL_ENERGY: false,
    CORE_FRAGMENTS: false,
    LORE_SHARDS: false,
};

export function createWallet(snap?: WalletSnapshot): Wallet {
    const balances: Record<Currency, number> = { ...ZERO, ...(snap?.balances ?? {}) };
    const ledgerEntries: LedgerEntry[] = snap?.ledger ? [...snap.ledger] : [];

    return {
        balance(c) {
            return balances[c] ?? 0;
        },
        earn(c, amount, reason) {
            if (amount <= 0) return;
            balances[c] = (balances[c] ?? 0) + amount;
            ledgerEntries.push({ currency: c, delta: amount, reason, ts: Date.now() });
        },
        spend(c, amount, reason) {
            if (amount <= 0) return true;
            const have = balances[c] ?? 0;
            if (have < amount) return false;
            balances[c] = have - amount;
            ledgerEntries.push({ currency: c, delta: -amount, reason, ts: Date.now() });
            return true;
        },
        transferable(c) {
            return TRANSFERABLE[c] ?? false;
        },
        ledger() {
            return [...ledgerEntries];
        },
        snapshot() {
            return { balances: { ...balances }, ledger: [...ledgerEntries] };
        },
    };
}

```

```
};  
}
```

- **[] Step 1.4: Run the test (expect pass)**

```
pnpm test:run src/wallet.test.ts
```

Expected: all 6 tests pass.

- **[] Step 1.5: Run the build (expect green)**

```
pnpm build
```

Expected: exit 0.

- **[] Step 1.6: Commit**

```
git add src/wallet.ts src/wallet.test.ts  
git commit -m "feat(wallet): currency abstraction with audit ledger (Phase 0)"
```

Task 2: Add wallet + season + class fields to GameState

Goal: Extend `GameState` and `initGame()` with the new fields so future tasks have a place to write.

Additive only — do not rename or remove anything.

Files:

- Modify: `src/game.ts`
- **[] Step 2.1: Read the existing `GameState` interface**

```
grep -n "interface GameState" src/game.ts
```

Note the line numbers for the next steps.

- **[] Step 2.2: Import wallet types at the top of `src/game.ts`**

Add this import alongside the existing imports near the top of the file:

```
import { createWallet, type Wallet, type WalletSnapshot, type Currency } from './wallet';
```

- **[] Step 2.3: Define the new state shapes (near the top, with other types)**

Add immediately above the `GameState` interface declaration:

```
// --- Phase 0 (Six-Month Grid foundation) ---
export interface SeasonState {
  seasonNumber: number;      // 0 in Phase 0 / Season 0
  startedAt: number;         // ms epoch when this season opened
  weekIndex: number;         // 1..8 (computed from startedAt; cached for save legibility)
  dailyStreak: number;       // consecutive days with a completed Daily Contract
  lastDailyClaim: number;    // ms epoch
}

export interface ClassProgression {
  primary: 'OPERATOR' | 'CORSAIR' | 'GHOST' | 'ARCHITECT';
  operator: { xp: number; mastery: number; skillsUnlocked: string[] };
  corsair?: { xp: number; mastery: number; skillsUnlocked: string[] }; // Season 1+
  ghost?: { xp: number; mastery: number; skillsUnlocked: string[] }; // Season 2+
  architect?: { xp: number; mastery: number; skillsUnlocked: string[] }; // Season 3+
}
```

- **[] Step 2.4: Add fields to `GameState`**

Inside the existing `GameState` interface (do NOT remove or rename anything else), append these fields:

```
// Phase 0 – Six-Month Grid foundation. Optional during migration; initGame() seeds them.
wallet?: WalletSnapshot;
seasonState?: SeasonState;
classProgression?: ClassProgression;
```

- **[] Step 2.5: Update `initGame()` to seed the new fields**

Locate `initGame()` (search for `export function initGame`). At the **end** of the returned object (before the closing brace), add the three new fields:

```
wallet: { balances: {
  DATA: 0, MEMORY_SHARDS: 0, ACCESS_KEYS: 0, SIGNAL_ENERGY: 0, CORE_FRAGMENTS: 0,
  LORE_SHARDS: 0,
}, ledger: [] },
seasonState: {
  seasonNumber: 0,
  startedAt: Date.now(),
  weekIndex: 1,
  dailyStreak: 0,
  lastDailyClaim: 0,
},
classProgression: {
  primary: 'OPERATOR',
  operator: { xp: 0, mastery: 0, skillsUnlocked: [] },
},
```

- **[] Step 2.6: Run the build**

```
pnpm build
```

Expected: exit 0. No typecheck errors.

- **[] Step 2.7: Commit**

```
git add src/game.ts
git commit -m "feat(game): add wallet/season/class fields to GameState (additive)"
```

Task 3: Season state module (TDD)

Goal: Pure functions for season math: current week from a start-timestamp, soft-reset detection, daily-streak update.

Files:

- Create: `src/seasons.ts`
- Create: `src/seasons.test.ts`

- **[] Step 3.1: Write the failing tests**

`src/seasons.test.ts`:

```

import { describe, it, expect } from 'vitest';
import {
  computeWeekIndex, isSeasonOver, updateDailyStreak, SEASON_LENGTH_MS, DAY_MS,
} from './seasons';

describe('Season math', () => {
  const start = Date.UTC(2026, 4, 1, 0, 0, 0); // 2026-05-01 00:00 UTC

  it('week 1 starts at the start-of-season', () => {
    expect(computeWeekIndex(start, start)).toBe(1);
  });

  it('week 1 still applies during day 6', () => {
    expect(computeWeekIndex(start, start + 6 * DAY_MS)).toBe(1);
  });

  it('week 2 begins on day 7', () => {
    expect(computeWeekIndex(start, start + 7 * DAY_MS)).toBe(2);
  });

  it('week 8 is the last week', () => {
    expect(computeWeekIndex(start, start + 49 * DAY_MS)).toBe(8);
  });

  it('season is over after 8 weeks (= 56 days)', () => {
    expect(isSeasonOver(start, start + 55 * DAY_MS)).toBe(false);
    expect(isSeasonOver(start, start + 56 * DAY_MS)).toBe(true);
  });

  it('SEASON_LENGTH_MS equals 56 days', () => {
    expect(SEASON_LENGTH_MS).toBe(56 * DAY_MS);
  });
});

describe('Daily streak', () => {
  const t0 = Date.UTC(2026, 4, 1, 12, 0, 0);

  it('first claim sets streak to 1', () => {
    const next = updateDailyStreak({ dailyStreak: 0, lastDailyClaim: 0 }, t0);
    expect(next.dailyStreak).toBe(1);
    expect(next.lastDailyClaim).toBe(t0);
  });

  it('claim on the next day continues the streak', () => {

```

```

    const next = updateDailyStreak({ dailyStreak: 5, lastDailyClaim: t0 }, t0 + DAY_MS);
    expect(next.dailyStreak).toBe(6);
  });

  it('claim within the same calendar day is a no-op', () => {
    const next = updateDailyStreak({ dailyStreak: 5, lastDailyClaim: t0 }, t0 + 1_000);
    expect(next.dailyStreak).toBe(5);
    expect(next.lastDailyClaim).toBe(t0);
  });

  it('claim after a 48h+ gap resets the streak to 1', () => {
    const next = updateDailyStreak({ dailyStreak: 9, lastDailyClaim: t0 }, t0 + 3 * DAY_MS);
    expect(next.dailyStreak).toBe(1);
  });
});

```

- **[] Step 3.2: Run tests (expect fail)**

```
pnpm test:run src/seasons.test.ts
```

Expected: FAIL — Cannot find module './seasons'.

- **[] Step 3.3: Implement `src/seasons.ts`**

```

/**
 * Season math – pure functions. Per the spec: 8-week soft-reset cadence, UTC daily resets.
 * Daily streak resets if more than 1 calendar day (UTC) is skipped; multiple claims in the
 * same UTC day are no-ops.
 */

export const DAY_MS = 24 * 60 * 60 * 1000;
export const SEASON_LENGTH_MS = 56 * DAY_MS; // 8 weeks

/** Returns 1..8 if within the season, else the clamped final week (8) once the season has run
out. */
export function computeWeekIndex(startedAt: number, now: number): number {
  const elapsed = Math.max(0, now - startedAt);
  const week = Math.floor(elapsed / (7 * DAY_MS)) + 1;
  return Math.min(8, Math.max(1, week));
}

export function isSeasonOver(startedAt: number, now: number): boolean {
  return now - startedAt >= SEASON_LENGTH_MS;
}

function utcDay(ts: number): number {
  return Math.floor(ts / DAY_MS);
}

/** Pure: returns the next (dailyStreak, lastDailyClaim) given the last-claim and the now-
instant. */
export function updateDailyStreak(
  prev: { dailyStreak: number; lastDailyClaim: number },
  now: number,
): { dailyStreak: number; lastDailyClaim: number } {
  if (prev.lastDailyClaim === 0) {
    return { dailyStreak: 1, lastDailyClaim: now };
  }

  const lastDay = utcDay(prev.lastDailyClaim);
  const nowDay = utcDay(now);
  if (nowDay === lastDay) {
    return prev; // already claimed today
  }
  if (nowDay === lastDay + 1) {
    return { dailyStreak: prev.dailyStreak + 1, lastDailyClaim: now };
  }
  return { dailyStreak: 1, lastDailyClaim: now };
}

```

- ☐ **Step 3.4: Run tests (expect pass)**

```
pnpm test:run src/seasons.test.ts
```

Expected: all 10 tests pass.

- ☐ **Step 3.5: Build**

```
pnpm build
```

Expected: exit 0.

- ☐ **Step 3.6: Commit**

```
git add src/seasons.ts src/seasons.test.ts
git commit -m "feat(seasons): pure season math + daily streak (Phase 0)"
```

Task 4: Save migration v2 (TDD — regression-critical)

Goal: Bump the save version to 2. Old v1 saves load **without breaking** — the loader fills in the new wallet/seasonState/classProgression fields from defaults. This is the highest-risk change in Phase 0; tests must lock it down.

Files:

- Modify: `src/save.ts`
- Create: `src/save.test.ts`

- ☐ **Step 4.1: Read the existing save shape**

```
grep -n "VERSION\|SaveBlob\|loadGame\|function migrate" src/save.ts
```

Confirm the v1 layout. The migration must NOT change the storage key or the legacy slot semantics.

- ☐ **Step 4.2: Write the failing tests**

```
src/save.test.ts:
```



```

import { describe, it, expect, beforeEach, vi } from 'vitest';
import { initGame } from './game';

// We test the migration helper directly. Importing dynamically so the localStorage stub is
// set up.
import { migrateSaveBlob } from './save';

describe('save migration', () => {
  it('a fresh v2 blob round-trips unchanged', () => {
    const fresh = { version: 2, continued: false, game: initGame() };
    const migrated = migrateSaveBlob(fresh);
    expect(migrated.version).toBe(2);
    expect(migrated.game.wallet).toBeDefined();
    expect(migrated.game.seasonState).toBeDefined();
    expect(migrated.game.classProgression).toBeDefined();
  });

  it('a v1 blob without wallet is migrated: wallet is seeded with zeros', () => {
    const v1Game = initGame();
    // Pretend v1 didn't have wallet/seasonState/classProgression
    delete (v1Game as { wallet?: unknown }).wallet;
    delete (v1Game as { seasonState?: unknown }).seasonState;
    delete (v1Game as { classProgression?: unknown }).classProgression;
    const blob = { version: 1, continued: true, game: v1Game };

    const migrated = migrateSaveBlob(blob);
    expect(migrated.version).toBe(2);
    expect(migrated.game.wallet).toBeDefined();
    expect(migrated.game.wallet?.balances.DATA).toBe(0);
    expect(migrated.game.seasonState?.seasonNumber).toBe(0);
    expect(migrated.game.classProgression?.primary).toBe('OPERATOR');
    expect(migrated.continued).toBe(true); // existing flag preserved
  });

  it('a v1 blob with a DATA balance migrates it into the wallet (preserves progress)', () => {
    const v1Game = initGame();
    // simulate a legacy in-state currency field (if any) – the migration must
    // adopt it as the wallet balance if present, never zero it out.
    delete (v1Game as { wallet?: unknown }).wallet;
    (v1Game as { data?: number }).data = 137;

    const migrated = migrateSaveBlob({ version: 1, continued: false, game: v1Game });
    expect(migrated.game.wallet?.balances.DATA).toBe(137);
  });
});

```

```
it('an unknown future version is left intact (forward-compat: do not destroy)', () => {
  const futureGame = initGame();
  const blob = { version: 99, continued: false, game: futureGame } as unknown as {
    version: number; continued: boolean; game: ReturnType<typeof initGame>;
  };
  const migrated = migrateSaveBlob(blob);
  expect(migrated.version).toBe(99);
  expect(migrated.game).toBe(futureGame);
});
});
```

- [] **Step 4.3: Run the tests (expect fail)**

```
pnpm test:run src/save.test.ts
```

Expected: FAIL — `migrateSaveBlob` is not exported (or `save.ts` has no migration helper to import).

- [] **Step 4.4: Add `migrateSaveBlob` and bump version in `src/save.ts`**

Open `src/save.ts`. At the top of the file, find:

```
const VERSION = 1;
```

Change it to:

```
const VERSION = 2;
```

Then, **above** the `interface SaveBlob` declaration, add the migration helper as an exported function (it is exported so tests can call it directly — keeping it pure simplifies testing and runtime use alike):

```

import type { GameState } from './game';

/**
 * Pure migration: takes any historical SaveBlob and returns a v2-shaped blob.
 * Additive only – never destroys data. A legacy `state.data: number` (if any)
 * is moved into `wallet.balances.DATA` so currency progress is preserved.
 *
 * Exported for direct testing.
 */
export function migrateSaveBlob<T extends { version: number; continued: boolean; game:
GameState }>(
  blob: T,
): T {
  // Forward-compat: never touch unknown future versions.
  if (blob.version > 2) return blob;

  const game = blob.game as GameState & { data?: number };

  if (!game.wallet) {
    const legacyData = typeof game.data === 'number' ? game.data : 0;
    game.wallet = {
      balances: {
        DATA: legacyData,
        MEMORY_SHARDS: 0,
        ACCESS_KEYS: 0,
        SIGNAL_ENERGY: 0,
        CORE_FRAGMENTS: 0,
        LORE_SHARDS: 0,
      },
      ledger: [],
    };
  }

  if (!game.seasonState) {
    game.seasonState = {
      seasonNumber: 0,
      startedAt: Date.now(),
      weekIndex: 1,
      dailyStreak: 0,
      lastDailyClaim: 0,
    };
  }

  if (!game.classProgression) {

```

```

game.classProgression = {
  primary: 'OPERATOR',
  operator: { xp: 0, mastery: 0, skillsUnlocked: [] },
};
}

return { ...blob, version: 2 };
}

```

- **[] Step 4.5: Wire `migrateSaveBlob` into the load path**

Find the function that loads from `localStorage` (search for `JSON.parse` on `KEY`). After parsing, before returning, route through `migrateSaveBlob`. Example shape (adapt to actual code — the existing loader may be named differently):

```

// inside the existing load function, after the raw blob is parsed
const parsed = JSON.parse(raw) as SaveBlob;
const migrated = migrateSaveBlob(parsed);
// then proceed with the existing merge-into-initGame() logic, using migrated.game

```

- **[] Step 4.6: Run the test suite (expect pass)**

```
pnpm test:run
```

Expected: all save + earlier tests pass.

- **[] Step 4.7: Manual smoke — old save loads green**

```
pnpm dev
```

In the browser devtools console (existing session, with a legacy v1 save in `localStorage`), reload the page. The mission should resume at the saved mission. Confirm:

```
JSON.parse(localStorage.getItem('neva:save:v1')).version === 2
```

After reload it should now be `2` (re-saved post-migration).

Stop the dev server (Ctrl+C).

- **[] Step 4.8: Build**

```
pnpm build
```

Expected: exit 0.

- **[] Step 4.9: Commit**

```
git add src/save.ts src/save.test.ts
git commit -m "feat(save): additive v2 migration for wallet/season/class fields"
```

Task 5: Event archetypes + types (TDD)

Goal: Type-safe definitions of the four event archetypes. Pure validators that accept/reject an event payload by archetype.

Files:

- Create: `src/events/types.ts`
- Create: `src/events/archetypes.ts`
- Create: `src/events/archetypes.test.ts`
- **[] Step 5.1: Write the types**

`src/events/types.ts`:

```
import type { Currency } from '../wallet';

export type Archetype = 'BREACH' | 'ANOMALY' | 'FACTION_OP' | 'TIDE';

export type EventCadence = 'DAILY' | 'WEEKLY' | 'TIDE';

export interface Reward {
  currency: Currency;
  amount: number;
}

export interface GameEvent {
  id: string; // server-generated UUID-ish
  archetype: Archetype;
  cadence: EventCadence;
  seasonNumber: number;
  title: string;
  brief: string; // 1-2 sentence player-facing brief
  targetNodeIds?: number[]; // for BREACH/ANOMALY (Sector A02 indices)
  durationMs: number;
  startsAt: number; // ms epoch
  endsAt: number; // ms epoch
  rewards: Reward[];
  payload?: Record<string, unknown>; // archetype-specific extra data
}
```

- [] **Step 5.2: Write the failing tests**

`src/events/archetypes.test.ts` :

```
import { describe, it, expect } from 'vitest';
import { validateEvent } from './archetypes';
import type { GameEvent } from './types';

const base: GameEvent = {
  id: 'evt-1',
  archetype: 'BREACH',
  cadence: 'DAILY',
  seasonNumber: 0,
  title: 'Breach the Archive',
  brief: 'A short breach run.',
  targetNodeIds: [42],
  durationMs: 15 * 60 * 1000,
  startsAt: 1_700_000_000_000,
  endsAt: 1_700_000_000_000 + 24 * 60 * 60 * 1000,
  rewards: [{ currency: 'DATA', amount: 10 }],
};

describe('validateEvent', () => {
  it('accepts a well-formed BREACH event', () => {
    expect(validateEvent(base).ok).toBe(true);
  });

  it('rejects a BREACH with no targetNodeIds', () => {
    const e = { ...base, targetNodeIds: undefined };
    expect(validateEvent(e).ok).toBe(false);
  });

  it('rejects an ANOMALY without targetNodeIds', () => {
    const e = { ...base, archetype: 'ANOMALY', targetNodeIds: undefined } as GameEvent;
    expect(validateEvent(e).ok).toBe(false);
  });

  it('accepts a TIDE with no targetNodeIds', () => {
    const e = { ...base, archetype: 'TIDE', cadence: 'TIDE', targetNodeIds: undefined } as GameEvent;
    expect(validateEvent(e).ok).toBe(true);
  });

  it('rejects endsAt <= startsAt', () => {
    const e = { ...base, endsAt: base.startsAt };
    expect(validateEvent(e).ok).toBe(false);
  });
});
```

```
it('rejects rewards with non-positive amounts', () => {
  const e = { ...base, rewards: [{ currency: 'DATA' as const, amount: 0 }] };
  expect(validateEvent(e).ok).toBe(false);
});
});
```

- **[] Step 5.3: Run the tests (expect fail)**

```
pnpm test:run src/events/archetypes.test.ts
```

Expected: FAIL — Cannot find module './archetypes'.

- **[] Step 5.4: Implement `src/events/archetypes.ts`**

```
import type { GameEvent } from './types';

export type Validation = { ok: true } | { ok: false; reason: string };

export function validateEvent(e: GameEvent): Validation {
  if (!e.id) return { ok: false, reason: 'missing id' };
  if (!e.title) return { ok: false, reason: 'missing title' };
  if (e.endsAt <= e.startsAt) return { ok: false, reason: 'endsAt must be > startsAt' };

  if (e.archetype === 'BREACH' || e.archetype === 'ANOMALY') {
    if (!e.targetNodeIds || e.targetNodeIds.length === 0) {
      return { ok: false, reason: `${e.archetype} requires targetNodeIds` };
    }
  }

  if (e.rewards.length === 0) return { ok: false, reason: 'no rewards' };
  for (const r of e.rewards) {
    if (r.amount <= 0) return { ok: false, reason: 'reward amounts must be > 0' };
  }

  return { ok: true };
}
```

- **[] Step 5.5: Run the tests (expect pass)**

```
pnpm test:run src/events/archetypes.test.ts
```

Expected: all 6 tests pass.

- **[] Step 5.6: Build + commit**


```
pnpm build
git add src/events/types.ts src/events/archetypes.ts src/events/archetypes.test.ts
git commit -m "feat(events): archetype types + validator (Phase 0)"
```

Task 6: Backend event store

Goal: A JSON-file-backed store under `server/` for active events + per-profile claim records. Follows the pattern of `server/waitlist-store.mjs` (zero deps).

Files:

- Create: `server/events-store.mjs`
- **[] Step 6.1: Read the existing JSON-store pattern**

```
head -60 server/json-store.mjs
head -60 server/waitlist-store.mjs
```

Confirm that `json-store.mjs` provides a `loadJson` / `saveJson` helper. The new store will reuse it.

- **[] Step 6.2: Create `server/events-store.mjs`**

```

/**
 * Event store – JSON-file-backed list of active events and per-profile claim records.
 * Zero dependencies; mirrors the pattern of waitlist-store.mjs.
 *
 * On-disk layout (under `data/events.json`):
 * { events: [{ id, archetype, cadence, ... }], claims: { [profileId]: [eventId, ...] } }
 */
import { loadJson, saveJson } from './json-store.mjs';

const FILE = 'events.json';
const SHAPE = { events: [], claims: {} };

function read() {
  const raw = loadJson(FILE, SHAPE);
  return {
    events: Array.isArray(raw.events) ? raw.events : [],
    claims: raw.claims && typeof raw.claims === 'object' ? raw.claims : {},
  };
}

export function listActiveEvents(now = Date.now()) {
  const { events } = read();
  return events.filter((e) => e.startsAt <= now && now < e.endsAt);
}

export function getEventById(id) {
  const { events } = read();
  return events.find((e) => e.id === id) ?? null;
}

export function addEvent(event) {
  const state = read();
  state.events.push(event);
  saveJson(FILE, state);
  return event;
}

export function deleteEvent(id) {
  const state = read();
  const before = state.events.length;
  state.events = state.events.filter((e) => e.id !== id);
  saveJson(FILE, state);
  return before !== state.events.length;
}

```

```

export function hasClaimed(profileId, eventId) {
  const { claims } = read();
  return (claims[profileId] ?? []).includes(eventId);
}

/** Idempotent: a second claim returns { ok: true, alreadyClaimed: true } without re-issuing
rewards. */
export function recordClaim(profileId, eventId) {
  const state = read();
  const list = state.claims[profileId] ?? [];
  if (list.includes(eventId)) return { ok: true, alreadyClaimed: true };
  list.push(eventId);
  state.claims[profileId] = list;
  saveJson(FILE, state);
  return { ok: true, alreadyClaimed: false };
}

export function eventStats() {
  const { events, claims } = read();
  const claimTotal = Object.values(claims).reduce((acc, l) => acc + l.length, 0);
  return { eventCount: events.length, claimTotal };
}

```

- [] **Step 6.3: Smoke-test from a Node REPL**

```

node --input-type=module -e "
import { addEvent, listActiveEvents, recordClaim, hasClaimed } from './server/events-
store.mjs';
const now = Date.now();
addEvent({
  id: 'evt-test', archetype: 'BREACH', cadence: 'DAILY', seasonNumber: 0,
  title: 'Test', brief: 'b', targetNodeIds:[1],
  durationMs: 60000, startsAt: now - 1000, endsAt: now + 60000,
  rewards: [{currency:'DATA', amount: 5}]
});
console.log('active:', listActiveEvents().length);
console.log('claim:', recordClaim('profA','evt-test'));
console.log('claim2:', recordClaim('profA','evt-test'));
console.log('hasClaimed:', hasClaimed('profA','evt-test'));
"

```

Expected output:

```
active: 1
claim: { ok: true, alreadyClaimed: false }
claim2: { ok: true, alreadyClaimed: true }
hasClaimed: true
```

- **[] Step 6.4: Clean up the test event**

```
node --input-type=module -e "import { deleteEvent } from './server/events-store.mjs';
deleteEvent('evt-test');"

```

- **[] Step 6.5: Commit**

```
git add server/events-store.mjs data/events.json 2>/dev/null; git add server/events-store.mjs
git commit -m "feat(server): JSON-file event store (Phase 0)"

```

(The `data/events.json` file may or may not exist depending on whether json-store auto-creates it. If it does and is in `.gitignore`, leave it alone.)

Task 7: Backend event routes

Goal: Expose `/api/events/active`, `/api/events/claim`, `/api/admin/events` (admin-gated CRUD) via the existing `server/index.mjs` dispatcher.

Files:

- Modify: `server/index.mjs`
- **[] Step 7.1: Read the existing route dispatcher**

```
grep -n "POST\|GET\|/api/" server/index.mjs | head -40

```

Identify the pattern used (likely a `switch` on `req.method` + `req.url`). New routes follow the same pattern.

- **[] Step 7.2: Import the event store at the top of `server/index.mjs`**

Add alongside other imports:

```
import { listActiveEvents, addEvent, deleteEvent, recordClaim, eventStats } from './events-
store.mjs';

```

- **[] Step 7.3: Add the three route branches**

Inside the existing request handler, add these branches (use the file's existing JSON-response helper for consistency — if it has one named e.g. `sendJson`):

```

// --- Six-Month Grid: event routes (Phase 0) ---

// GET /api/events/active → public; returns list of currently-active events
if (req.method === 'GET' && url.pathname === '/api/events/active') {
  const events = listActiveEvents();
  return sendJson(res, 200, { ok: true, events });
}

// POST /api/events/claim → body: { profileId, eventId }
if (req.method === 'POST' && url.pathname === '/api/events/claim') {
  const body = await readJsonBody(req); // existing helper
  const profileId = String(body?.profileId ?? '').slice(0, 64);
  const eventId = String(body?.eventId ?? '').slice(0, 64);
  if (!profileId || !eventId) {
    return sendJson(res, 400, { ok: false, error: 'profileId and eventId required' });
  }
  const result = recordClaim(profileId, eventId);
  return sendJson(res, 200, result);
}

// POST /api/admin/events → admin-gated CRUD (create / delete / list)
if (req.method === 'POST' && url.pathname === '/api/admin/events') {
  if (!ADMIN_SECRET || !checkAdminAuth(req)) { // existing helper / pattern
    return sendJson(res, 403, { ok: false, error: 'forbidden' });
  }
  const body = await readJsonBody(req);
  const op = String(body?.op ?? '');
  if (op === 'create') {
    const ev = body?.event;
    if (!ev?.id) return sendJson(res, 400, { ok: false, error: 'event.id required' });
    addEvent(ev);
    return sendJson(res, 200, { ok: true, event: ev });
  }
  if (op === 'delete') {
    const id = String(body?.id ?? '');
    const removed = deleteEvent(id);
    return sendJson(res, 200, { ok: true, removed });
  }
  if (op === 'stats') {
    return sendJson(res, 200, { ok: true, stats: eventStats() });
  }
  return sendJson(res, 400, { ok: false, error: 'unknown op' });
}

```

Engineer note: The function names `sendJson`, `readJsonBody`, `checkAdminAuth` are placeholders for whatever the existing file uses. Inspect `server/index.mjs` first and call the actual helpers — do not introduce new ones. The Phase-8 admin endpoints already have an `admin-auth` pattern; reuse it verbatim.

- **[] Step 7.4: Smoke-test the routes (server up)**

In one terminal:

```
pnpm server
```

In another:

```
# Active should be [] (assuming no seeded events)
curl -s http://localhost:8787/api/events/active

# Claim without seeded event → just records the claim
curl -s -X POST http://localhost:8787/api/events/claim \
  -H 'Content-Type: application/json' \
  -d '{"profileId":"p1","eventId":"evt-1"}'
```

Expected: HTTP 200 JSON for both.

Stop the server (Ctrl+C).

- **[] Step 7.5: Build (ensures no TS impact, though backend is JS)**

```
pnpm build
```

Expected: exit 0.

- **[] Step 7.6: Commit**

```
git add server/index.mjs
git commit -m "feat(server): /api/events/* routes (active, claim, admin)"
```

Task 8: Frontend events service

Goal: A tiny client-side fetcher that talks to `/api/events/active` and `/api/events/claim`. The dev Vite proxy already forwards `/api` to `:8787`.

Files:

- Create: `src/events/service.ts`
- Create: `src/events/service.test.ts`

- [] **Step 8.1: Write the failing test**

`src/events/service.test.ts:`

```

import { describe, it, expect, vi, beforeEach, afterEach } from 'vitest';
import { fetchActiveEvents, claimEvent } from './service';

const originalFetch = globalThis.fetch;

beforeEach(() => {
  vi.useFakeTimers();
});
afterEach(() => {
  globalThis.fetch = originalFetch;
  vi.useRealTimers();
});

describe('events service', () => {
  it('fetchActiveEvents returns the event list on 200', async () => {
    globalThis.fetch = vi.fn().mockResolvedValue(
      new Response(JSON.stringify({ ok: true, events: [{ id: 'a' }, { id: 'b' }] }),
      { status: 200 }
    );
    const list = await fetchActiveEvents();
    expect(list).toHaveLength(2);
  });

  it('fetchActiveEvents returns [] on network error', async () => {
    globalThis.fetch = vi.fn().mockRejectedValue(new Error('offline'));
    const list = await fetchActiveEvents();
    expect(list).toEqual([]);
  });

  it('claimEvent posts profileId and eventId and returns the server result', async () => {
    const fetchMock = vi.fn().mockResolvedValue(
      new Response(JSON.stringify({ ok: true, alreadyClaimed: false }), { status: 200 }
    );
    globalThis.fetch = fetchMock;
    const res = await claimEvent('profA', 'evt-x');
    expect(res).toEqual({ ok: true, alreadyClaimed: false });
    const call = fetchMock.mock.calls[0];
    expect(call[0]).toBe('/api/events/claim');
    expect(JSON.parse((call[1] as RequestInit).body as string)).toEqual({
      profileId: 'profA', eventId: 'evt-x',
    });
  });
});

```


- **[] Step 8.2: Run (expect fail)**

```
pnpm test:run src/events/service.test.ts
```

Expected: FAIL — Cannot find module './service'.

- **[] Step 8.3: Implement `src/events/service.ts`**

```
import type { GameEvent } from './types';

export async function fetchActiveEvents(): Promise<GameEvent[]> {
  try {
    const res = await fetch('/api/events/active');
    if (!res.ok) return [];
    const data = (await res.json()) as { ok: boolean; events?: GameEvent[] };
    return data.ok && data.events ? data.events : [];
  } catch {
    return [];
  }
}

export interface ClaimResult {
  ok: boolean;
  alreadyClaimed?: boolean;
  error?: string;
}

export async function claimEvent(profileId: string, eventId: string): Promise<ClaimResult> {
  try {
    const res = await fetch('/api/events/claim', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ profileId, eventId }),
    });
    if (!res.ok) return { ok: false, error: `http ${res.status}` };
    return (await res.json()) as ClaimResult;
  } catch (e) {
    return { ok: false, error: String(e) };
  }
}
```

- **[] Step 8.4: Run (expect pass)**

```
pnpm test:run src/events/service.test.ts
```

Expected: 3 tests pass.

- **[] Step 8.5: Build + commit**

```
pnpm build
git add src/events/service.ts src/events/service.test.ts
git commit -m "feat(events): client-side fetch/claim service"
```

Task 9: DailyContractCard component

Goal: A small HUD card at top-right of the screen showing the active Daily Contract (if any). Click → opens an inspection panel placeholder. Click "Claim" after a contract is completed → posts to `/api/events/claim`.

For Phase 0, "completed" is signaled by a placeholder button (real gameplay-integration ships in Phase 1). The card just needs to render, claim, and disappear when claimed.

Files:

- Create: `src/components/DailyContractCard.tsx`
- **[] Step 9.1: Implement the component**

```
import { useEffect, useState } from 'react';
import { fetchActiveEvents, claimEvent } from '../events/service';
import type { GameEvent } from '../events/types';

const cardStyle: React.CSSProperties = {
  position: 'fixed',
  top: 16,
  right: 16,
  width: 280,
  padding: 12,
  background: 'rgba(10,14,39,0.85)',
  color: '#d8e8ff',
  border: '1px solid #6a4aaa',
  borderRadius: 8,
  fontFamily: 'SF Mono, Menlo, Consolas, monospace',
  fontSize: 12,
  zIndex: 1000,
  pointerEvents: 'auto',
};

const titleStyle: React.CSSProperties = {
  fontSize: 11, letterSpacing: 2, color: '#b8c8ff', marginBottom: 4,
};

const briefStyle: React.CSSProperties = { marginBottom: 8, lineHeight: 1.4 };

const btnStyle: React.CSSProperties = {
  width: '100%', padding: '6px 8px', background: '#2a0050', color: '#fff',
  border: 'none', borderRadius: 4, cursor: 'pointer', fontSize: 11,
};

interface Props {
  profileId: string;
}

export function DailyContractCard({ profileId }: Props) {
  const [event, setEvent] = useState<GameEvent | null>(null);
  const [claimed, setClaimed] = useState(false);
  const [busy, setBusy] = useState(false);

  useEffect(() => {
    let cancelled = false;
    fetchActiveEvents().then((list) => {
      if (cancelled) return;
    });
  });
}
```

```

    const daily = list.find((e) => e.cadence === 'DAILY') ?? null;
    setEvent(daily);
  });
  return () => {
    cancelled = true;
  };
}, []);

if (!event || claimed) return null;

return (
  <div style={cardStyle} data-testid="daily-contract-card">
    <div style={titleStyle}>DAILY CONTRACT</div>
    <div style={{ marginBottom: 6, fontWeight: 600 }}>{event.title}</div>
    <div style={briefStyle}>{event.brief}</div>
    <button
      type="button"
      style={btnStyle}
      disabled={busy}
      onClick={async () => {
        setBusy(true);
        const res = await claimEvent(profileId, event.id);
        setBusy(false);
        if (res.ok) setClaimed(true);
      }}
    >
      {busy ? 'CLAIMING...' : 'CLAIM REWARD'}
    </button>
  </div>
);
}

```

- **[] Step 9.2: Build (TS check)**

```
pnpm build
```

Expected: exit 0.

- **[] Step 9.3: Commit**

```

git add src/components/DailyContractCard.tsx
git commit -m "feat(ui): DailyContractCard HUD component (Phase 0)"

```

Task 10: SeasonTracker component

Goal: A small top-left HUD strip showing weeks 1–8 with the current week highlighted.

Files:

- Create: `src/components/SeasonTracker.tsx`
- **[] Step 10.1: Implement the component**

```
import { computeWeekIndex } from '../seasons';

interface Props {
  seasonNumber: number;
  startedAt: number;
}

const wrapStyle: React.CSSProperties = {
  position: 'fixed', top: 16, left: 16,
  display: 'flex', alignItems: 'center', gap: 8,
  fontFamily: 'SF Mono, Menlo, Consolas, monospace',
  color: '#d8e8ff', fontSize: 11, letterSpacing: 2,
  zIndex: 1000, pointerEvents: 'none',
};

const labelStyle: React.CSSProperties = { opacity: 0.7 };

const cellStyle = (active: boolean): React.CSSProperties => ({
  width: 16, height: 6, borderRadius: 2,
  background: active ? '#b88aff' : 'rgba(120,140,200,0.25)',
  boxShadow: active ? '0 0 6px #b88aff' : 'none',
});

export function SeasonTracker({ seasonNumber, startedAt }: Props) {
  const week = computeWeekIndex(startedAt, Date.now());
  return (
    <div style={wrapStyle} data-testid="season-tracker">
      <span style={labelStyle}>S{seasonNumber} · W{week}</span>
      {Array.from({ length: 8 }, (_, i) => (
        <div key={i} style={cellStyle(i + 1 <= week)} />
      ))}
    </div>
  );
}
```

- **[] Step 10.2: Build + commit**

```
pnpm build
git add src/components/SeasonTracker.tsx
git commit -m "feat(ui): SeasonTracker HUD component (Phase 0)"
```

Task 11: Wire DailyContractCard + SeasonTracker into GameApp

Goal: Mount the two HUD components in `src/GameApp.tsx`. They read from current game state.

Files:

- Modify: `src/GameApp.tsx`
- **[] Step 11.1: Identify the mount site**

```
grep -n "return\|<>" src/GameApp.tsx | head -20
```

Find the top-level returned JSX (usually wrapped in a fragment or a top-level `<div>`).

- **[] Step 11.2: Import the two components and any state hook used to access GameState**

Add at the top of `src/GameApp.tsx`:

```
import { DailyContractCard } from './components/DailyContractCard';
import { SeasonTracker } from './components/SeasonTracker';
```

- **[] Step 11.3: Add them inside the returned JSX**

Place them as siblings of the existing top-level HUD elements, AFTER the Canvas (so they overlay):

```
<SeasonTracker
  seasonNumber={state.seasonState?.seasonNumber ?? 0}
  startedAt={state.seasonState?.startedAt ?? Date.now()}
/>
<DailyContractCard profileId={profile.id} />
```

Engineer note: `state` and `profile` are placeholder names. Use whatever the file already binds the game state and the player-profile to. If a profile-id is not readily available, default to `'local'` for Phase 0 — Phase 1 will introduce profile-scoped claims.

- **[] Step 11.4: Manual smoke test**

```
pnpm server &
pnpm dev
```

Open `http://localhost:5173` (or the Vite-reported port). Confirm:

1. SeasonTracker visible at top-left with "S0 · W1" and 1/8 cells lit.
2. DailyContractCard *not yet* visible (no events seeded — that's Step 11.5).

Stop both.

- [] **Step 11.5: Seed a Daily event and re-test the card**

In a terminal, set the admin secret and seed an event:

```
export NEVA_ADMIN_SECRET="phase0-test"
pnpm server &

# Seed
NOW=$(node -e "console.log(Date.now())")
END=$(node -e "console.log(Date.now() + 24*3600*1000)")
curl -s -X POST http://localhost:8787/api/admin/events \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $NEVA_ADMIN_SECRET" \
  -d '{"op": "create", "event": {"id": "daily-1", "archetype": "BREACH", "cadence": "DAILY", "seasonNumber": 0, "title": "Surface Breach", "brief": "Decode 3 surface nodes.", "targetNodeIds": [7, 12, 19], "durationMs": 900000, "startsAt": $NOW, "endsAt": $END, "rewards": [{"currency": "DATA", "amount": 10}]}}'
```

Reload the browser. The DailyContractCard should now render with "Surface Breach". Click **CLAIM REWARD** → card disappears. Repeat click on a manually re-rendered card → server returns `alreadyClaimed: true`.

Stop the server.

- [] **Step 11.6: Build + commit**

```
pnpm build
git add src/GameApp.tsx
git commit -m "feat(ui): mount SeasonTracker + DailyContractCard in GameApp"
```

Task 12: Admin Event Editor (minimal HTML)

Goal: A static HTML page served from the server at `/admin/events` for creating and deleting events without curl. Gated by entering the `NEVA_ADMIN_SECRET` in the page (sent as Authorization header).

Files:

- Create: `server/admin-events.html`
- Modify: `server/index.mjs` (add the static route)

- [] **Step 12.1: Create** `server/admin-events.html`


```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<title>NEVA Network – Event Admin</title>
<style>
  body { font-family: -apple-system, sans-serif; background: #0a0e27; color: #d8e8ff; margin:
0; padding: 24px; }
  h1 { color: #b88aff; }
  input, textarea, button, select {
    background: #1a1147; color: #d8e8ff; border: 1px solid #6a4aaa;
    padding: 6px 8px; border-radius: 4px; margin: 4px 0; font-family: inherit;
  }
  textarea { width: 600px; height: 220px; font-family: SF Mono, monospace; font-size: 12px; }
  button { cursor: pointer; }
  .row { margin: 8px 0; }
  pre { background: #1a1147; padding: 8px; border-radius: 4px; max-width: 600px; overflow-x:
auto; }
</style>
</head>
<body>
<h1>NEVA NETWORK · EVENT ADMIN</h1>

<div class="row">
  <label>Admin secret:</label><br/>
  <input id="secret" type="password" size="40" placeholder="NEVA_ADMIN_SECRET" />
</div>

<h2>Create event (JSON)</h2>
<textarea id="json">{
  "id": "weekly-1",
  "archetype": "ANOMALY",
  "cadence": "WEEKLY",
  "seasonNumber": 0,
  "title": "Pulse from the Deep",
  "brief": "Trace the source of a 7-node pulse anomaly.",
  "targetNodeIds": [42, 73, 91, 104, 118, 137, 158],
  "durationMs": 7200000,
  "startsAt": 0,
  "endsAt": 0,
  "rewards": [{"currency":"LORE_SHARDS","amount":5}]
}</textarea>
<div class="row">
  <button id="create">Create</button>
```

```

    <button id="stats">Stats</button>
  </div>

  <h2>Delete event</h2>
  <input id="delId" placeholder="event id" />
  <button id="delete">Delete</button>

  <h2>Result</h2>
  <pre id="out">—</pre>

  <script>
const out = document.getElementById('out');
const secret = () => document.getElementById('secret').value;

async function call(op, payload = {}) {
  const res = await fetch('/api/admin/events', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + secret() },
    body: JSON.stringify({ op, ...payload }),
  });
  out.textContent = await res.text();
}

document.getElementById('create').onclick = async () => {
  const txt = document.getElementById('json').value;
  let ev;
  try { ev = JSON.parse(txt); } catch { out.textContent = 'Invalid JSON'; return; }
  const now = Date.now();
  if (!ev.startsAt) ev.startsAt = now;
  if (!ev.endsAt) ev.endsAt = now + (ev.cadence === 'DAILY' ? 24 : 168) * 3600 * 1000;
  await call('create', { event: ev });
};

document.getElementById('delete').onclick = () =>
  call('delete', { id: document.getElementById('delId').value });

document.getElementById('stats').onclick = () => call('stats');
</script>
</body>
</html>

```

- [] **Step 12.2: Add a static route in `server/index.mjs`**

Add this branch alongside the other static-file handling (or near the top of the request handler):

```
if (req.method === 'GET' && url.pathname === '/admin/events') {
  const html = readFileSync(join(dirname(fileURLToPath(import.meta.url)), 'admin-
events.html'), 'utf8');
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end(html);
  return;
}
```

(The `readFileSync`, `dirname`, `fileURLToPath`, and `join` imports already exist at the top of `server/index.mjs` per the read in Task 7.)

- **[] Step 12.3: Smoke-test**

```
export NEVA_ADMIN_SECRET="phase0-test"
pnpm server &
```

Open `http://localhost:8787/admin/events` in the browser. Enter the secret. Click **Create** with the default JSON → `out` shows the created event. Click **Stats** → confirms count incremented.

Stop the server.

- **[] Step 12.4: Commit**

```
git add server/admin-events.html server/index.mjs
git commit -m "feat(server): /admin/events minimal editor"
```

Task 13: Integration regression + Phase 0 gate verification

Goal: Confirm M00–M20 still plays, all Phase 0 gates pass, and no forbidden deps were added.

Files: none (verification only)

- **[] Step 13.1: Full test suite**

```
pnpm test:run
```

Expected: all tests pass (wallet, seasons, save, archetypes, service).

- **[] Step 13.2: Full build**

```
pnpm build
```

Expected: exit 0.

- **[] Step 13.3: Lint**

```
pnpm lint
```

Expected: exit 0.

- [] **Step 13.4: Grep audit — no Web3 dependencies introduced**

```
node -e "
const pkg = require('./package.json');
const deps = {...pkg.dependencies, ...pkg.devDependencies};
const banned =
['ethers','web3','wagmi','viem','@walletconnect/','@solana/','@reown/','rainbowkit','starknet'
];
const hits = Object.keys(deps).filter(d => banned.some(b => d.includes(b)));
console.log(hits.length ? 'FAIL — Web3 deps found: ' + hits.join(', ') : 'OK — no Web3 deps');
process.exit(hits.length ? 1 : 0);
"
```

Expected output: OK — no Web3 deps .

- [] **Step 13.5: Grep audit — no any or @ts-ignore introduced in Phase 0 files**

```
grep -nE "\\bany\\b|@ts-ignore|@ts-nocheck" \
  src/wallet.ts src/seasons.ts src/events/*.ts src/components/DailyContractCard.tsx
src/components/SeasonTracker.tsx 2>&1 \
  | grep -v "as { wallet?:" || echo "OK — no new any / ts-ignore"
```

Expected: OK — no new any / ts-ignore (the lone `as { wallet?: unknown }` cast in the save migration is acceptable — it's narrowly scoped at the migration boundary).

- [] **Step 13.6: Manual playthrough — Mission 00**

```
pnpm server &
pnpm dev
```

Open the game, start a fresh save (or use the existing one). Confirm:

1. SeasonTracker top-left shows S0 · W1.
2. Mission 00 tutorial plays through to completion (9 guided beats).
3. After Mission 00, mission narrative resumes per the existing chain.

Stop both.

Note: You do not need to play through all 20 missions to mark this step done — confirm M00 + the next mission start, which exercises save migration, reducer, and the new HUD overlays together. A full M00→M20 playthrough is the Phase 1 gate.

- **[] Step 13.7: Final commit (if any cleanup needed)**

```
git status
# If anything changed during smoke testing (e.g. data/events.json regenerated), commit or
revert as appropriate.
```

Task 14: Phase 0 complete — document, then offer Phase 1 plan

Goal: Close out Phase 0 with a short note in `CURRENT_PROJECT_STATE.md` (or wherever phase-status lives), then signal "Phase 0 done, ready for Phase 1 plan."

Files:

- Modify: `CURRENT_PROJECT_STATE.md` (search for "Phase" / "Status" section; append "Six-Month Grid Phase 0: complete").

- **[] Step 14.1: Update the status note**

Add a one-paragraph entry under the project status table:

```
- Six-Month Grid Phase 0 (Foundation) - Complete (2026-XX-XX). Adds vitest harness,
  wallet abstraction, season state, save migration v2, event archetypes, backend event store +
  routes, frontend events service, DailyContractCard, SeasonTracker, and the /admin/events
  page.
  M00-M20 unchanged. No Web3 deps. Ready for Phase 1 - Season 0 "First Cipher" content
  authoring.
```

- **[] Step 14.2: Commit**

```
git add CURRENT_PROJECT_STATE.md
git commit -m "docs: mark Six-Month Grid Phase 0 complete"
```

- **[] Step 14.3: Phase 0 is done.**

Phase 1 (Season 0 content authoring + NEVA Core event-drafter + leaderboard + Pattern Lens permanent unlock) gets its own plan once we agree to proceed.

Phase 0 /goal directive (for autonomous execution)

/goal Implement Phase 0 of the Six-Month Grid plan in
docs/superpowers/plans/2026-05-28-six-month-grid-phase-0.md.

Work through Tasks 0-14 in order. Per task, follow the TDD steps
exactly (failing test → impl → passing test → commit).

Done when:

- All 14 tasks are committed (`git log --oneline | head -20` shows them).
- `pnpm test:run` exits 0 with at least 25 passing tests.
- `pnpm build` exits 0.
- `pnpm lint` exits 0.
- `node -e "const p=require('./package.json'); const d={...p.dependencies,...p.devDependencies}; const b=['ethers','web3','wagmi','viem','@walletconnect/','@solana/','@reown/','rainbowkit','starknet']; const h=Object.keys(d).filter(x=>b.some(y=>x.includes(y))); process.exit(h.length)"` exits 0.
- Manual smoke: dev server starts, Mission 00 plays through, SeasonTracker visible, seeded DailyContractCard renders and claim button works.
- `docs/superpowers/specs/2026-05-28-data-grid-game-design.md` is unchanged (spec is read-only).
- `src/missions.ts`, `src/network.ts`, `src/sectorGen.ts` are unchanged.

Constraints:

- Do not modify Missions 00-20 logic.
- Do not introduce any Web3 / token / wallet-connect / smart-contract module.
- All currency mutations must route through wallet.ts (audit by grep).
- No new `any` or `@ts-ignore` outside the one allowed save-migration cast.
- Backend stays zero-dep until SQLite is justified (Phase 1+).
- Each task is its own commit.

Stop after 60 turns max.

Self-Review (run after the plan is written)

Spec coverage — every Phase 0 milestone gate in spec §7 is implemented:

Spec gate	Task
1. <code>wallet.ts</code> routes all currency mutations	Task 1 + Task 2
2. <code>seasons.ts</code> + save migration v2	Tasks 3 + 4
3. <code>/api/events/active</code> returns at least one daily + weekly	Tasks 6 + 7 + Step 11.5
4. Admin Event Editor at <code>/admin/events</code>	Task 12
5. Daily Contract HUD renders + completes + claims	Tasks 8 + 9 + 11
6. Weekly Anomaly opens in Sector A02 + completes + claims	Task 7 (route covers weekly) + admin can seed; full UI integration deferred to Phase 1 — flagged in Phase 1 backlog
7. Season tracker UI shows weeks 1–8	Task 10
8. <code>pnpm build</code> green	Task 13 gate
9. M00–M20 still plays	Task 13 Step 13.6

Placeholder scan — none found.

Type consistency — `Currency`, `WalletSnapshot`, `LedgerEntry`, `SeasonState`, `ClassProgression`, `GameEvent`, `Archetype`, `EventCadence`, `Reward`, `ClaimResult` are defined exactly once and referenced consistently. Type aliases match across `wallet.ts`, `seasons.ts`, `events/types.ts`, `events/service.ts`, and the save migration.

Scope check — Phase 0 only. Subsequent phases will get their own plan docs.

Part IV — Future Phase Outlines

These outlines are **deliberately not full implementation plans**. Each phase gets its own detailed plan only when the previous phase ships — so playtest feedback shapes the next design. Writing all seven plans up front would be wasted work.

Each outline below specifies: **goal**, **duration**, ****new content/mechanics introduced, engine modules added, dependencies on prior phases****, and **high-level task buckets** (what the eventual plan will cover).

§4.1 Phase 1 — Season 0 Launch (engine + content meet)

Goal: Take the Phase 0 engine (built but empty) and load the Season 0 Content Bible into it. After Phase 1, players can play Season 0 end-to-end across 8 weeks — Operator class only.

Duration: ~8 weeks of work.

Status of inputs:

-  Phase 0 engine plan written.
-  Season 0 Content Bible written.
-  Phase 1 plan itself NOT yet written (this is the next document to write).

Content shipped (from Content Bible):

- 4 Factions live (Archive, Black Loop, Quiet Pattern, Severance) with reputation
- 8 Puzzle types live (Caesar, Frequency, Pattern-Walk, Symbol-Sub, Graph, Light Physics, NEVA Whisper, Echo Overlay)
- 7 Weekly Anomalies authored (`WA-S0-01` ... `07`)
- 12 Daily Contract templates seeded (`DC-01` ... `12`)
- Season 0 Tentpole (`TP-S0`) playable end-to-end
- NEVA whisper library + lie-rate mechanism
- Pattern Lens permanent unlock at finale
- Lore page system

New engine modules:

- `src/puzzles/types.ts` + per-puzzle file (Caesar, Frequency, Pattern-Walk, NEVA, etc.)

- `src/puzzles/registry.ts` — weighted random picker
- `src/factions/` — store + reputation reducer
- `src/featureFlags.ts` — `PATTERN_LENS_T1` etc.
- `server/data/seasons/s0/` — JSON content: `weeklies.json`, `daily-templates.json`, `tentpole.json`, `beats.json`
- `server/data/whispers.json` — NEVA library
- `server/scheduler.mjs` — daily/weekly event spawn cron
- `server/ai-event-drafter.mjs` — NEVA Core LLM event-template filler (uses existing `OPENAI_API_KEY`)
- `src/components/FactionRepGauge.tsx`
- `src/components/AnomalyPanel.tsx`
- `src/components/TentpoleStageView.tsx`
- `src/components/PatternLensOverlay.tsx`

Task buckets the Phase 1 plan will cover (15–20 tasks):

1. Puzzle engine — Caesar (P-CES) TDD
2. Puzzle engine — Frequency (P-FRQ) TDD
3. Puzzle engine — Pattern Walk (P-PWK) TDD
4. Puzzle engine — Symbol Sub (P-SUB) TDD
5. Puzzle engine — Graph (P-GRT) TDD
6. Puzzle engine — Light Physics (P-LPH) TDD
7. Puzzle engine — NEVA Whisper (P-NWI) + lie-rate TDD
8. Puzzle engine — Echo Overlay (P-ECO) TDD
9. Puzzle registry + weighted picker TDD
10. Faction store + reputation reducer TDD
11. Seed Season 0 weeklies (WA-S0-01..07) into Event Store
12. Seed Season 0 daily templates (DC-01..12)
13. Wire daily scheduler — picks template, generates today's contract via NEVA Core drafter
14. Wire weekly Anomaly into Sector A02 procedural spawn
15. Story beats (week 1..8) emitted on weekly boundaries
16. Tentpole (TP-S0) staged delivery — 7 stages
17. Soft anti-grind caps (in claim path)
18. NEVA whisper library + audio tell system
19. Lore page unlock system + Pattern Lens flag
20. Regression — M00–M20 still plays, Season 0 fully playable

Phase 1 `/goal` directive (preview):

/goal Implement Season 0 "First Cipher" content on the Phase 0 engine, per docs/superpowers/specs/2026-05-28-content-bible-season-0.md.

Done when:

- All 8 puzzle engines pass their unit tests.
- 4 factions store + reputation reducer pass tests.
- Server seeds WA-S0-01..07 + DC-01..12 + TP-S0 on first boot.
- A full Mission 00 → Tentpole completion is playable end-to-end.
- Pattern Lens unlocks on TP-S0 stage 7 and persists across sessions.
- M00-M20 still plays without regression.
- `pnpm test:run` exits 0 with 60+ tests passing.
- `pnpm build` + `pnpm lint` exit 0.
- No new Web3 deps.

Constraints:

- Every content piece uses its canonical ID from the Content Bible.
- Wallet routes all currency mutations.
- NEVA Core drafter is admin-approved before any published content.
- No code edits to existing M00-M20 logic.

Stop after 90 turns max.

Risks specific to Phase 1:

- Puzzle UX variance — some puzzles harder to make feel "good" than others. Mitigation: ship Caesar / Frequency / Pattern-Walk first (Bible §3.9 high-weight); polish NEVA/Echo last.
- Anomaly pacing — first playtests may reveal weekly stages are too long. Mitigation: Bible's stage durations are *targets*, not contracts.
- Faction balance — reputation curves may feel slow. Mitigation: tune drop rates in claim path before public alpha.

§4.2 Phase 2 — Season 1 "Black Routes" (Corsair class)

Goal: Add the second class (Corsair) and Season 1's 8-week arc. After

Phase 2, players have two classes to choose from, four+ months of content total.

Duration: ~8 weeks of work.

Dependencies: Phase 0 + Phase 1 fully shipped.

Content shipped (NEW Content Bible to be written: Season 1):

- Corsair class — **BREACH-AND-RUN** signature mechanic

- Black Credits currency live
- 5–7 new puzzle types specific to breach gameplay (timed/route)
- 7 new Weekly Anomalies (`WA-S1-01` ... `07`)
- 12 new Daily Contract templates (`DC-S1-01` ... `12`)
- New Tentpole (`TP-S1`): the Black Loop's deeper agenda revealed
- ICE (Intrusion Countermeasures) subsystem + visual design
- Soft season-reset: Sector A02 regenerates, Season 0 lore archived to "Sector A01 expanded archive"

New engine modules:

- `src/classes/corsair/` — class-specific reducer, tools, signature ability "Cold Trail"
- `src/ice/` — Intrusion Countermeasures (timers, alarms, trace risk)
- `src/components/BreachTimerHud.tsx`
- `src/components/ClassSwitcher.tsx` (multi-class support enters here)
- `src/blackMarket/` — NPC fencing vendor
- New skill tree UI

Task buckets the Phase 2 plan will cover (~20 tasks):

1. Class abstraction in `GameState` (multi-class support, primary + dabbles)
2. Corsair class state + reducer TDD
3. ICE system — alarm clock, trace, lockdown TDD
4. Breach timer HUD
5. Black Credits earn/spend in wallet
6. Spoofer / Ghost-tail / Snip tools (3 new puzzle types)
7. Class Switcher UI
8. Black Market vendor — fence stolen data
9. NEW Content Bible — Season 1 (separate doc, ~700 lines like S0)
10. Seed Season 1 weeklies + dailies
11. New Tentpole (TP-S1) — 7 stages
12. Cold Trail signature ability (Mastery 4 unlock)
13. Soft season-reset migration (Sector A02 regenerates)
14. Returning-player flow ("welcome back, here's what changed")
15. Cross-class interaction: Corsair steals → Operator decrypts faster (rep split)
16. Leaderboard for Season 1 Anomalies
17. Regression on all prior content

Open design questions to answer before Phase 2 starts:

- Should ICE be deterministic (puzzle) or stochastic (real-time)? Bible can lean either way; needs playtest signal from Season 0.
- Should Corsair share the same node graph or have a "shadow grid" overlay?
- How does multi-class XP feel — punish dabblers, or treat all classes as one pool?

§4.3 Phase 3 — Season 2 "The Echo Below" (Ghost class)

Goal: Add the third class (Ghost) and Season 2's arc. After Phase 3, players have three classes; ~6 months of content total. **This is the season that unlocks the user's "6+ month engagement" goal.**

Duration: ~8 weeks of work.

Dependencies: Phases 0–2 fully shipped.

Content shipped (NEW Content Bible: Season 2):

- Ghost class — `POSSESSION` mechanic
- Coherence currency (account-wide, not per-season)
- Possession graph overlay system
- 5–7 new puzzle types around resonance / spread mechanics
- 7 new Weekly Anomalies (`WA-S2-01` ... `07`)
- 12 new Daily Contract templates (`DC-S2-01` ... `12`)
- Tentpole (`TP-S2`): the SELF confrontation — meet your own mirror operator
- New visual / audio language for ghost mode (chromatic aberration, distant hum)

New engine modules:

- `src/classes/ghost/` — possession state, tendril graph, coherence budget
- `src/ghostGraph/` — non-Euclidean overlay (player's "presence" across nodes)
- `src/components/GhostOverlay.tsx` (R3F shader)
- `src/components/CoherenceMeter.tsx`
- "Singularity" Mastery-4 ability — permanent personal anchor on a cluster

Task buckets the Phase 3 plan will cover (~22 tasks):

1. Possession graph data structure TDD
2. Coherence budget + decay rules TDD
3. Possession overlay shader (R3F)
4. Tendril / Echo / Resonate tools (3 new mechanics)
5. Ghost class state in `GameState`
6. NEW Content Bible — Season 2
7. Seed Season 2 weeklies + dailies
8. Tentpole TP-S2 — the SELF confrontation
9. Singularity ability + permanent cluster anchor system
10. New audio motif (Quiet Pattern bowl resonance bleeds into Ghost gameplay)
11. Cross-class interaction: Ghost can infest Architect base (deferred Phase 4 wiring)
12. Memory-fragment lore unlocks tied to coherence

Open design questions:

- Coherence as account-wide currency: does this risk a "must grind to be powerful" treadmill? Need to design caps now.
- The "SELF confrontation" is emotionally heavy — needs a playtest with non-developer testers before public.

§4.4 Phase 4 — Season 3 "Bastion" (Architect class) — the biggest phase

Goal: Add the fourth and final base class (Architect) and Season 3's

arc. After Phase 4, the game is feature-complete on the class axis — all four classes live + async multiplayer raid replays.

Duration: ~12 weeks of work (50% longer than other phases due to base-building scope).

Dependencies: Phases 0–3 fully shipped.

Content shipped (NEW Content Bible: Season 3):

- Architect class — BUILD-AND-DEFEND mechanic
- Grid Power currency (account-wide; passive yield)
- Private grid editor (player's own sector)
- Cipher Wall / ICE Trap / Yield Tap building primitives
- Async raid system — other players' Corsairs/Ghosts attempt your base offline; replay on your next login
- 7 new Weekly Anomalies (WA-S3-01 ... 07)
- 12 new Daily Contract templates (DC-S3-01 ... 12)
- Tentpole (TP-S3): build a public landmark that other players visit
- "Bastion Mode" Mastery-4 — your base broadcasts as a notable landmark with a small visit reward

New engine modules:

- `src/classes/architect/` — class state, build system, defense rules
- `src/baseBuilder/` — placement grid, validation, encryption tiers
- `src/raids/` — async raid replay system (record on attacker side, deliver on victim's next login)
- `src/components/BaseEditor.tsx` (substantial UI)
- `src/components/RaidReplay.tsx`
- `server/raids/` — raid request matching, offline yield calculator

Task buckets the Phase 4 plan will cover (~28 tasks):

1. Base placement grid TDD
2. Build primitives: Wall, Trap, Tap TDD
3. Encryption tiers (defense math) TDD

4. Offline yield calculator TDD (with time-decay caps)
5. Async raid recording — Corsair side
6. Async raid replay — Architect side
7. Raid matching — find a victim per a difficulty curve
8. NEW Content Bible — Season 3
9. Base editor UI (big — multi-task)
10. Public landmark broadcast (Bastion mode)
11. Visit reward — visiting an Architect base grants small Grid Power
12. Permanent persistence — Architect bases survive seasonal soft-resets
13. Anti-griefing: cap raids-per-day per Architect; protect new players
14. Cross-class loop: Ghost can possess Architect node → defense special case
15. Regression on all prior content

Open design questions (these all need answers before Phase 4 starts):

- Persistence model — Architect bases never reset. How does this fit the PoE soft-reset model? Tentative answer: base + Grid Power persist; Architect's Season-only modifiers do not.
- Raid difficulty matching — without players we have no playerbase. Initial waves are AI-generated Corsairs.
- Anti-griefing for new Architects — 30-day grace period? Capped raids? Needs concrete numbers.
- Mobile fit — base editor needs touch input; defer mobile to Phase 5+?

§4.5 Phase 5 — Sandbox AI Faction Layer (stretch — toward true EVE)

Goal: Replace static factions with **simulated AI faction agents** that have agendas, conflicts, and emergent behavior. This is the long-term ambition (the "Approach A" from the Design Spec, finally reachable after the 4-class base ships).

Duration: 12+ weeks of work.

Dependencies: Phases 0–4 fully shipped + a playerbase to playtest with.

Content shipped:

- 4 Faction AI agents — each with: goals, resource pools, decision policy
- Inter-faction conflict simulation (territory, contested sectors, alliances)
- Procedural events emerge from faction state (not hand-authored)
- UGC tools (Phase 5b stretch): players can submit content drafts
- Steam release candidate

New engine modules:

- `src/factionAi/` — agent state, policy, decision loop (runs server-side, tick-based)
- `src/factionAi/policies/` — one per faction's behavior profile
- `server/worldTick.mjs` — periodic world simulation (every 4h?)
- `server/ugcReview.mjs` — moderation queue for player-submitted content

Task buckets:

- This phase is mostly research + iteration, not pre-planned tasks.
- Likely written as multiple sub-phases (5a, 5b, 5c) once we have a playerbase.

Open question: Is Phase 5 even desirable, or should we keep authored factions and just scale them? Decide after Phase 4 ships.

§4.6 Phase 6 — Token Activation (blocked by Sharia + legal review)

Goal: Activate the token-ready WalletProvider abstraction with a real on-chain wallet provider. Utility + earn + governance roles (per the user's long-term intent).

Duration: Indeterminate. Gated by external review.

Pre-conditions (per Design Spec §5):

-  Wallet abstraction in place (Phase 0 — done)
-  Independent Sharia review (gharar / maisir / qimar analysis)
-  Independent legal review (US + EU + KSA + UAE securities law)
-  Whitepaper draft + audit-firm pre-screen
-  Clear utility-vs-security classification

Task buckets (when unblocked):

1. On-chain WalletProvider implementation (single file swap)
2. Token contract deployment + audit
3. Read-only on-chain proofs of ledger reasons
4. Treasury smart contract (utility spend escrow)
5. Governance voting contract (on-chain rep + class mastery as voting weight)
6. Earn-tier emissions schedule (P2E balanced against soft caps from Phase 0)
7. KYC integration if legal review requires
8. Public token launch comms

Open question: Is this still desirable after Phases 0–5 ship? Many token-game projects have failed; the abstraction lets you decide later without rewriting code. Re-evaluate when the time comes.

Part V — Status Tracker

§5.1 Phase Status Table

Phase	What it ships	Engine plan	Content Bible	Built?
0	Foundation engine (wallet, seasons, events, HUD)	✅ Written, ready to <code>/goal</code>	n/a	🕒
1	Season 0 playable (Operator class)	❌ Not yet written	✅ Written	🕒
2	Season 1 — Corsair class	❌ Not yet written	❌ Not yet written	🕒
3	Season 2 — Ghost class	❌ Not yet written	❌ Not yet written	🕒
4	Season 3 — Architect class	❌ Not yet written	❌ Not yet written	🕒
5	Sandbox AI factions	❌ Not yet written	n/a (systemic)	🕒
6	Token activation	Indef. deferred	n/a	🕒

§5.2 Documents owed (in writing order)

1. **Phase 1 implementation plan** — the bridge that takes Bible + Engine and produces a `/goal`-runnable plan for Season 0. **This is the next document to write.**
2. **Season 1 Content Bible** — after Phase 0 + Phase 1 ship.
3. **Phase 2 implementation plan** — after Season 1 Bible.
4. **Season 2 Content Bible** — after Phase 2 ships.
5. **Phase 3 implementation plan** — after Season 2 Bible.
6. **Season 3 Content Bible** — after Phase 3 ships.
7. **Phase 4 implementation plan** — after Season 3 Bible (largest plan).
8. **Phase 5 plan** — only if pursuing the EVE-true sandbox stretch.
9. **Phase 6 plan** — gated by Sharia + legal review.

§5.3 Open decisions still pending

- **Faction names** — confirmed: Archive / Black Loop / Quiet Pattern / Severance (locked in Content Bible §2).
- **Daily reset time** — UTC midnight (locked in Content Bible §6 generation rule).
- **Streak grace** — 24h tolerance + 1 weekly grace (per Design Spec Open Questions).
- **Anomaly leaderboard scope** — global asynchronous (per Design Spec recommendation).
- **Steam/itch release timing** — Phase 5, after Architect class ships.

§5.4 What unblocks each next step

To do this...	We need...	Status
Build the engine (Phase 0)	Phase 0 plan	✓ Have it
Build Season 0 (Phase 1)	Phase 1 plan	✗ Need to write
Build Season 1 (Phase 2)	Phase 0 + 1 shipped + Season 1 Bible	✗ Both upstream
Confirm 6-month engagement works	At least Phase 0 + 1 + 2 shipped + 6-week playtest	✗ Far out
Activate token (Phase 6)	Sharia + legal reviews complete	✗ External blocker

§5.5 Recommended next action

Write the Phase 1 implementation plan. It is the only missing artifact between "engine built" and "playable Season 0 in your hands." Without it, running `/goal` Phase 0 produces a working engine that has nothing in it. After Phase 1 is plan-written and `/goal`-executed, the next decision point is whether to immediately write Phase 2 or pause and playtest Season 0 with real users (recommended).

End of master plan. Documents below this point are the unmodified contents of the originals; they remain individually maintained in their respective folders and were assembled here for single-document review.